

EMMA — Guida completa

Assistente personale self-hosted · versione 1, solo testo

31 agosto 2026

Indice

1	Introduzione	3
2	Capitolo 1 — Preparazione dell'ambiente	4
2.1	1.1 Quale Ubuntu, e perché	4
2.2	1.2 Verifica del punto di partenza	4
2.3	1.3 Aggiornamento del sistema	5
2.4	1.4 Pacchetti di sistema	5
2.5	1.5 Utente di sistema dedicato	5
2.6	1.6 Directory di installazione e permessi	6
2.7	1.7 Il secondo disco per i backup	6
2.8	1.8 Firewall	8
2.9	1.9 Le credenziali che ti servono	8
3	Capitolo 2 — Architettura	9
3.1	2.1 Il quadro d'insieme	9
3.2	2.2 Il percorso di un messaggio	9
3.3	2.3 Pattern adapter: perché core/ non sa cosa sia Telegram	10
3.4	2.4 Router agentico: il ciclo che oggi gira a vuoto	10
3.5	2.5 Memoria dietro interfaccia	11
3.6	2.6 Resilienza: tre livelli	12
3.7	2.7 Perché FastAPI se non c'è niente da esporre	12
3.8	2.8 Il provider e il modello	13
4	Capitolo 3 — Implementazione	14
4.1	3.1 La mappa	14
4.2	3.2 config.py — la configurazione	14
4.3	3.3 core/memory.py — la memoria	15
4.4	3.3bis core/tasks.py e tools/ — commissionare sviluppo	15
4.5	3.4 core/llm.py — il client del modello	17
4.6	3.5 core/router.py — l'orchestratore	18
4.7	3.6 adapters/telegram.py — il canale	18
4.8	3.7 main.py — l'avvio	18
4.9	3.8 tests/ — la suite	19
5	Capitolo 4 — Deploy	20
5.1	4.1 Il repository su GitHub	20
5.2	4.2 Scaricare il codice sul server	20
5.3	4.3 L'ambiente virtuale	20
5.4	4.4 Il file .env	21
5.5	4.5 Primo avvio a mano	21
5.6	4.6 Il servizio systemd	22
5.7	4.7 Il timer di backup	23
5.8	4.8 Riepilogo: cosa c'è ora sulla macchina	23
5.9	4.9 La coda di sviluppo	24
6	Capitolo 5 — Utilizzo	28

6.1	5.1 Il giorno per giorno	28
6.2	5.2 Cosa aspettarsi	28
6.3	5.3 Personalizzare il carattere	28
6.4	5.4 Quanto costa	29
6.5	5.5 Limiti noti della versione 1	29
6.6	5.6 Commissionare uno sviluppo	29
6.7	5.7 Azzerare la memoria	30
7	Capitolo 6 — Manutenzione	31
7.1	6.1 Leggere i log	31
7.2	6.2 La regola d'oro: prima il backup	32
7.3	6.3 Aggiornare il codice: PC di sviluppo → GitHub → server	32
7.4	6.4 Aggiornare le dipendenze bloccate	33
7.5	6.5 Backup della configurazione e della memoria	34
7.6	6.6 Ripristino	34
7.7	6.7 Problemi comuni	36
7.8	6.8 Calendario di manutenzione	39
8	Appendice A — Comandi di riferimento	40
9	Appendice B — Le variabili di .env	41
10	Appendice C — Dove guardare quando qualcosa non torna	42

Capitolo 1

Introduzione

Questa guida porta un server Ubuntu appena installato ad avere EMMA in funzione: un assistente personale con cui chatti da Telegram, che gira su hardware tuo e che parla con un modello linguistico attraverso l'API di Anthropic o quella di Groq, a scelta.

È il manuale completo di riferimento. Il `README.md` del repository è la guida rapida in inglese per chi vuole solo provarlo; questa guida è quella da seguire per installarlo davvero, capirlo e mantenerlo nel tempo. Seguendola dall'inizio alla fine non ti serve cercare informazioni altrove.

Cosa fa EMMA nella versione 1. Scrivi al tuo bot Telegram dal telefono. Un servizio Python sul tuo server riceve il messaggio, ci aggiunge la conversazione recente, chiede una risposta al modello e te la rimanda. Il bot risponde solo a te. Il modello si sceglie in configurazione: Claude tramite l'API di Anthropic, oppure Groq, che ha un piano gratuito. La conversazione è salvata su disco e sopravvive ai riavvii. Non c'è voce e non ci sono strumenti: è la fondazione, e tutto il resto arriverà sopra questa.

A chi si rivolge. A chi sa muoversi in un terminale Linux ma non dà nulla per scontato. Ogni comando è scritto per intero e ogni scelta è motivata, perché fra sei mesi vorrai sapere non solo *cosa* hai fatto ma *perché*.

Convenzioni.

- I comandi preceduti da `sudo` vanno eseguiti da un utente con privilegi amministrativi; gli altri dal tuo utente normale.
- Dove compare `/opt/emma`, è la directory di installazione: se la cambi, cambiala ovunque (nella guida e nei file in `systemd/`, dove le righe da toccare sono marcate con `# PATH:`).
- Dove compare `<tuo-account>`, sostituisci il tuo nome utente GitHub.
- I blocchi marcati **Verifica** dicono cosa devi vedere per essere sicuro che il passo sia andato a buon fine. Non saltarli: è così che si evita di scoprire un errore tre capitoli dopo.

I sei capitoli.

1. **Preparazione dell'ambiente** — dal server nudo al sistema pronto.
2. **Architettura** — com'è fatto EMMA e perché.
3. **Implementazione** — il codice, file per file.
4. **Deploy** — configurazione, servizio `systemd`, primo avvio.
5. **Utilizzo** — l'uso quotidiano dal telefono e i limiti noti.
6. **Manutenzione** — log, aggiornamenti, backup, ripristino, problemi comuni.

Capitolo 2

Capitolo 1 — Preparazione dell'ambiente

2.1 1.1 Quale Ubuntu, e perché

Scelta: Ubuntu Server 24.04 LTS (Noble Numbat).

Le LTS attualmente supportate sono la 24.04 (aprile 2024, supporto standard fino ad aprile 2029) e la 26.04, uscita nel 2026 e quindi con la finestra di supporto più lunga. Per questo progetto scelgo la 24.04 per tre motivi concreti:

- **è matura.** Ha alle spalle anni di point release: i bug di gioventù su driver, rete e installer sono risolti, e su un vecchio PC riciclato — che è esattamente il tuo caso — la compatibilità hardware già collaudata vale più di qualunque novità.
- **la documentazione di terze parti è enorme.** Quando qualcosa non funziona, cercando un messaggio d'errore trovi risposte scritte per la 24.04. Su una LTS appena uscita si finisce spesso a tradurre istruzioni pensate per la precedente.
- **ha Python 3.12,** che soddisfa con margine il requisito 3.11+ del progetto.

La 26.04 LTS è una scelta altrettanto legittima se preferisci il supporto più lungo: EMMA ci gira senza modifiche (Python 3.13, anch'esso compatibile). Tutti i comandi di questa guida valgono per entrambe. Quello che **non** consiglio è una release intermedia non-LTS: nove mesi di supporto significano un aggiornamento maggiore quasi ogni anno su una macchina che deve solo stare accesa e funzionare.

Scarica solo la variante Server, non la Desktop: niente ambiente grafico significa meno RAM occupata, meno pacchetti da aggiornare e meno superficie d'attacco. L'immagine è su <https://ubuntu.com/download/server>.

Durante l'installazione:

- crea il tuo utente personale (quello con cui farai ssh), non serve altro;
- **installa OpenSSH server** quando l'installer lo propone: è l'unico modo per amministrare la macchina senza monitor e tastiera attaccati;
- non installare snap aggiuntivi (Docker, Kubernetes e simili): non servono.

2.2 1.2 Verifica del punto di partenza

Collegati via SSH e guarda cosa hai:

```
ssh tuo-utente@indirizzo-del-server
```

```
lsb_release -a          # versione di Ubuntu
python3 --version        # deve essere >= 3.11
free -h                 # memoria disponibile
df -h /                  # spazio sul disco di sistema
ip -brief address        # indirizzi di rete
```

Verifica. `python3 --version` deve rispondere `Python 3.12.x` (o superiore) e `df -h /` deve mostrare almeno 5 GB liberi. EMMA occupa poche decine di MB, ma `virtualenv`, `log` e `backup` vogliono respiro.

2.3 1.3 Aggiornamento del sistema

Prima di installare qualunque cosa, allinea il sistema:

```
sudo apt update
sudo apt upgrade -y
sudo reboot          # solo se l'upgrade ha toccato il kernel
```

Se apt segnala che è richiesto un riavvio (** System restart required **), riavvia adesso: un kernel a metà strada è la causa più stupida di problemi successivi.

2.4 1.4 Pacchetti di sistema

Tutti dai repository ufficiali di Ubuntu. **Non aggiungiamo nessun PPA né repository esterno**, e vale la pena dire perché: ogni repository di terze parti è un soggetto che può pubblicare pacchetti sulla tua macchina con i privilegi di root, e va aggiornato e fidato per anni. Qui non serve: tutto ciò di cui EMMA ha bisogno è nei repository ufficiali, e le librerie Python arrivano da PyPI dentro un virtualenv isolato, senza toccare il Python di sistema.

```
sudo apt install -y \
    python3 \
    python3-venv \
    python3-pip \
    git \
    curl \
    ca-certificates \
    sqlite3 \
    tar \
    gzip
```

A cosa serve ciascuno:

Pacchetto	Perché
python3	l'interprete; già presente, lo elenchiamo per completezza
python3-venv	crea l'ambiente virtuale isolato del progetto
python3-pip	installa le dipendenze dentro quell'ambiente
git	scarica il codice e permette di tornare a una versione precedente
curl	verifiche manuali (l'endpoint /health, la connettività)
ca-certificates	certificati radice per le connessioni HTTPS verso Anthropic e Telegram
sqlite3	usato da scripts/backup.sh per lo snapshot consistente del database
tar, gzip	usati da scripts/backup.sh (di norma già installati)

Verifica.

```
python3 -m venv --help > /dev/null && echo "venv ok"
git --version
curl -sI https://api.anthropic.com | head -1
```

L'ultimo comando deve restituire una riga HTTP/...: qualunque risposta va bene, significa che il server raggiunge Internet in HTTPS. Se non risponde nulla, il problema è di rete o di DNS e va risolto prima di proseguire.

2.5 1.5 Utente di sistema dedicato

EMMA non deve girare come te, e tantomeno come root. Le va creato un utente apposta, **senza possibilità di login e senza privilegi**: se un giorno una skill avrà un bug o una dipendenza si rivelerà malevola, il danno resta confinato a ciò che quell'utente può toccare — cioè quasi nulla.

```
sudo useradd --system --create-home --home-dir /opt/emma --shell /usr/sbin/nologin emma
```

Cosa fanno le opzioni:

- `--system`: utente di servizio, con UID basso, escluso dalle liste di login;
- `--create-home --home-dir /opt/emma`: la home coincide con la directory di installazione, così il servizio ha un posto suo dove stare;
- `--shell /usr/sbin/nologin`: nessuno può fare login come emma, nemmeno con la password giusta, perché una password non esiste.

Verifica.

```
id emma
# uid=999(emma) gid=999(emma) groups=999(emma)
sudo -u emma whoami
# emma
```

2.6 1.6 Directory di installazione e permessi

```
sudo mkdir -p /opt/emma
sudo chown emma:emma /opt/emma
sudo chmod 750 /opt/emma
```

750 significa: emma legge e scrive, il gruppo emma legge, tutti gli altri utenti della macchina non vedono nemmeno il contenuto. Dato che dentro finirà il file `.env` con la tua chiave API, è il minimo.

Per lavorare comodamente aggiungiti al gruppo emma:

```
sudo usermod -aG emma $USER
newgrp emma          # applica il nuovo gruppo alla sessione corrente
```

Senza questo passaggio dovresti anteporre `sudo -u emma` a ogni comando dentro `/opt/emma`. Nota che `newgrp` vale solo per la shell corrente: alla prossima connessione SSH il gruppo sarà già attivo da solo.

2.7 1.7 Il secondo disco per i backup

Tu ragioni in termini di “disco D”, che su Windows è una lettera; su Linux un secondo disco fisico è un dispositivo da montare in un punto dell’albero delle directory. Il risultato è lo stesso — dati su un disco diverso da quello di sistema — ma la procedura è questa.

Se non hai un secondo disco, salta pure al capitolo 2: non devi configurare niente. `backup.sh` sceglie da sé la destinazione, con questa regola:

Situazione	Dove scrive
<code>/mnt/backup</code> è davvero un disco separato	<code>/mnt/backup/emma</code>
non c’è un secondo disco montato	<code>/var/backups/emma</code> , sul disco di sistema
hai impostato <code>BACKUP_DIR</code> nel <code>.env</code>	lì, comunque, senza discutere

Il backup **avviene sempre**: un archivio in un posto mediocre vale più di un archivio che non esiste. Quando ripiega sul disco di sistema lo dice nel log e nel manifesto, perché resta un compromesso — ti protegge dai tuoi errori, non dal guasto del disco.

Perché non scrivere in `/mnt/backup` quando il disco non è montato. Sembrerebbe funzionare, e invece riempirebbe il disco di sistema senza dirlo; peggio, il giorno in cui montassi davvero il disco quegli archivi sparirebbero sotto il punto di mount, continuando a occupare spazio che nessuno riesce più a vedere. Per questo lo script controlla che sia un filesystem separato, non che la directory esista.

2.7.1 1.7.1 Identificare il disco

```
lsblk -o NAME,SIZE,TYPE,FSTYPE,MOUNTPOINT,MODEL
```

Esempio di output:

```
NAME    SIZE TYPE FSTYPE MOUNTPOINT MODEL
sda     240G disk                KINGSTON_SA400
├─sda1   1G part vfat    /boot/efi
└─sda2 239G part ext4    /
sdb     500G disk                WDC_WD5000AAKX
```

Qui sda è il disco di sistema (contiene /) e sdb è il secondo disco, ancora senza filesystem e senza punto di mount.

Controlla due volte la lettera prima di proseguire: i comandi che seguono cancellano il contenuto del disco che indichi.

2.7.2 1.7.2 Partizionare e formattare

Attenzione: i due comandi seguenti distruggono tutti i dati su `/dev/sdb`. Se il disco contiene qualcosa che ti serve, copialo altrove prima.

```
sudo parted /dev/sdb --script mklabel gpt mkpart primary ext4 0% 100%
sudo mkfs.ext4 -L backup /dev/sdb1
```

ext4 è la scelta giusta qui: stabile, supportata ovunque, senza opzioni da capire. L'etichetta backup serve fra un attimo.

2.7.3 1.7.3 Montarlo in modo permanente

Un mount a mano sparisce al riavvio. Perché sia permanente va scritto in `/etc/fstab`, e va scritto usando l'**UUID** del filesystem, non `/dev/sdb1`: i nomi dei dispositivi possono cambiare fra un avvio e l'altro se aggiungi o togli un disco, l'UUID no.

```
sudo mkdir -p /mnt/backup
sudo blkid /dev/sdb1
# /dev/sdb1: LABEL="backup" UUID="1a2b3c4d-..." TYPE="ext4"
```

Copia l'UUID e aggiungi la riga a `/etc/fstab`:

```
sudo cp /etc/fstab /etc/fstab.bak          # rete di sicurezza
echo 'UUID=1a2b3c4d-... /mnt/backup ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab
```

L'opzione **nofail** è importante: senza di essa, se un giorno il disco si guasta o lo stacchi, il server non completa l'avvio e resta bloccato in emergency mode — con l'assistente spento per un problema di backup. Con **nofail** prosegue e il backup fallisce con un messaggio chiaro nei log, che è il comportamento giusto.

```
sudo systemctl daemon-reload
sudo mount -a
```

Verifica.

```
findmnt /mnt/backup
df -h /mnt/backup
```

Se `mount -a` non dà errori e `findmnt` mostra il disco, la riga di `fstab` è corretta e il montaggio sopravviverà al riavvio. Un errore qui va risolto adesso: un `fstab` sbagliato impedisce l'avvio della macchina.

2.7.4 1.7.4 La directory dei backup

```
sudo mkdir -p /mnt/backup/emma
sudo chown emma:emma /mnt/backup/emma
sudo chmod 700 /mnt/backup/emma
```


700 perché gli archivi contengono il `.env`, quindi la tua chiave API: solo l'utente emma deve poterli leggere. Questo percorso è quello che scriverai in `BACKUP_DIR` nel capitolo 4.

2.8 1.8 Firewall

EMMA **non espone nulla in ingresso**: parla con Telegram e con il provider LLM aprendo connessioni in uscita, e l'endpoint `/health` è in ascolto solo su `127.0.0.1`, raggiungibile unicamente dalla macchina stessa. Il firewall quindi non deve aprire niente per EMMA: deve solo lasciarti entrare in SSH.

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
```

Prima di dare `ufw enable`, assicurati che `sudo ufw allow OpenSSH` sia andato a buon fine. Attivare il firewall senza aver aperto SSH ti chiude fuori dalla macchina, e per rientrare serve monitor e tastiera.

Verifica.

```
sudo ufw status verbose
```

Devi vedere Default: deny (incoming), allow (outgoing) e una sola regola consentita, 22/tcp (OpenSSH). È esattamente la configurazione coerente con l'architettura: nessuna porta aperta, nessun webhook, nessun certificato da gestire.

2.9 1.9 Le credenziali che ti servono

Prima del deploy procurati questi tre valori. Tienili da parte: li scriverai nel `.env` al capitolo 4.

La chiave del provider LLM — una delle due, secondo quale userai.

Anthropic (a pagamento). Su <https://console.anthropic.com>, sezione *API Keys*, crea una chiave. Comincia con `sk-ant-`. Viene mostrata **una volta sola**: copiala subito. Ricorda anche di impostare un limite di spesa mensile nella sezione fatturazione — è la protezione più semplice contro una sorpresa.

Groq (piano gratuito). Su <https://console.groq.com> crea una chiave: comincia con `gsk_`. È l'opzione da scegliere se vuoi tenere la spesa a zero; in cambio i modelli disponibili dipendono dal piano e i limiti di richieste al minuto sono più stretti. Puoi cambiare provider in qualunque momento, è una riga nel `.env`.

Token del bot Telegram. Dal telefono, scrivi a [@BotFather](#):

1. `/newbot`
2. scegli un nome visualizzato (per esempio Emma)
3. scegli uno username che finisca per `bot` (per esempio `emma_assistant_bot`)

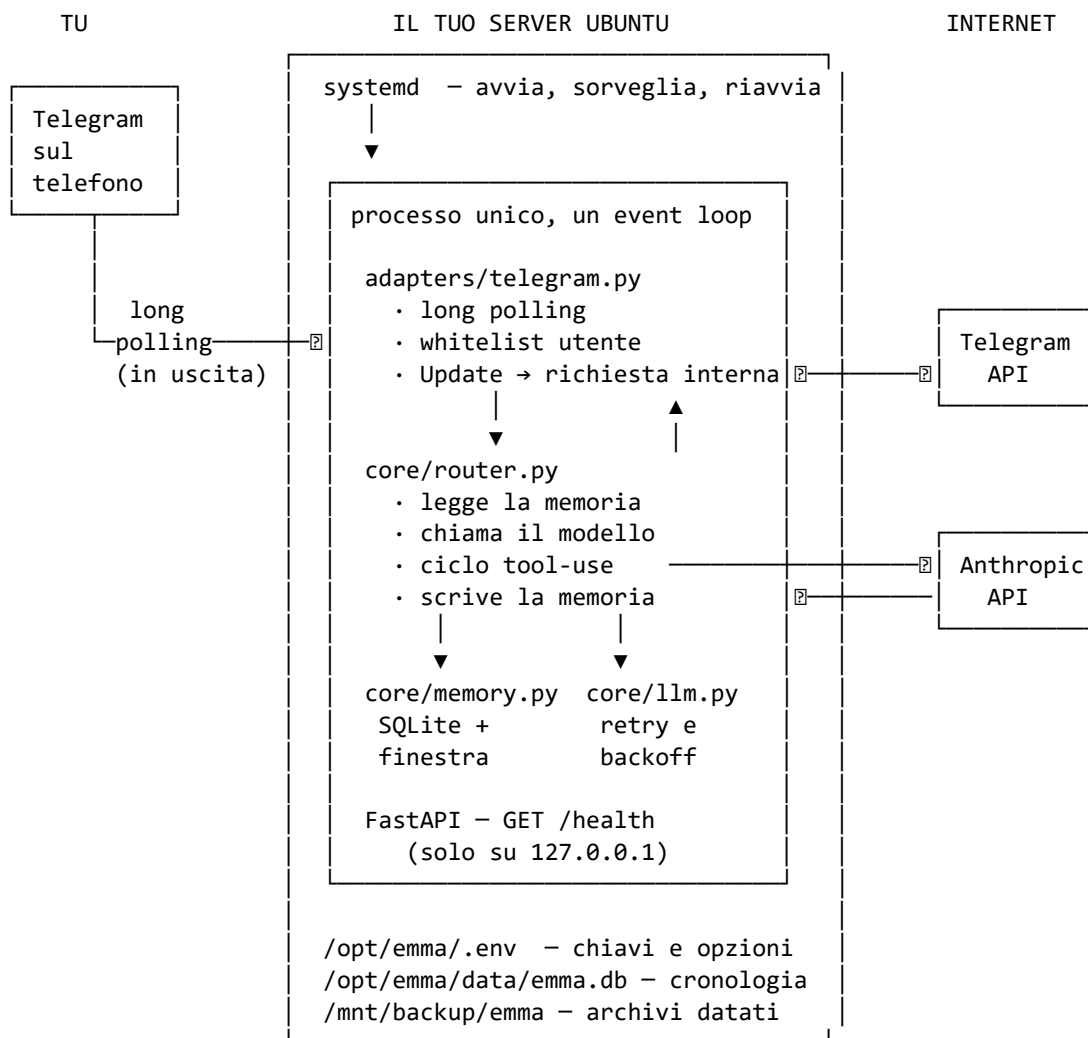
BotFather risponde con un token nella forma `123456789:AAH...`. È una **credenziale**: chi ce l'ha controlla il bot. Non finisce mai in Git, mai in un messaggio, mai in un log.

Il tuo user ID Telegram. Non è lo username con la chiocciola, è un numero. Scrivi a [@userinfobot](#): ti risponde con il tuo Id. Quel numero va in `TELEGRAM_ALLOWED_USER_ID` ed è ciò che fa rispondere il bot a te e a nessun altro.

Capitolo 3

Capitolo 2 — Architettura

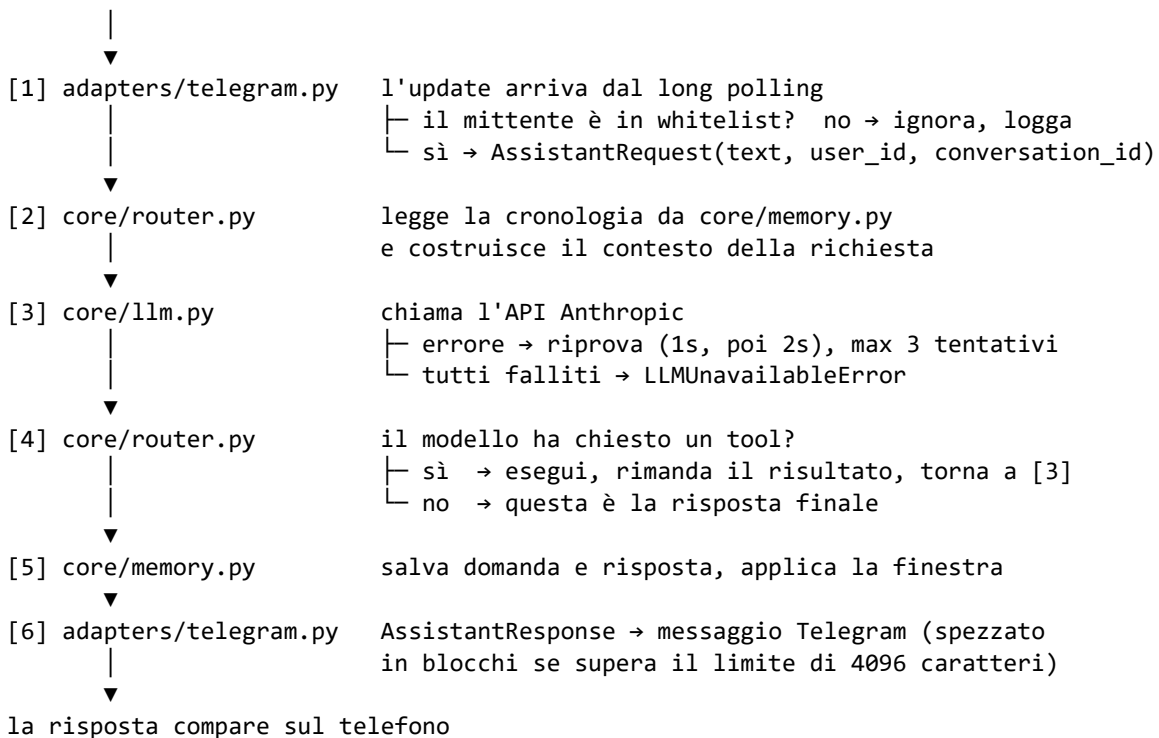
3.1 2.1 Il quadro d'insieme



Nessuna porta in ingresso. Tutte le frecce verso Internet partono da dentro.

3.2 2.2 Il percorso di un messaggio

tu scrivi "che tempo fa?"



Se [3] fallisce del tutto, il router non solleva l'errore: risponde con una frase che dice quale dei guasti è successo -- modello irraggiungibile, quota esaurita, risposta vuota, troppi giri di tool -- non salva nulla in memoria, e il processo resta vivo.

3.3 2.3 Pattern adapter: perché core/ non sa cosa sia Telegram

È la decisione strutturale più importante del progetto. La regola è una sola: **nessun file sotto core/ importa Telegram, e nessun concetto di Telegram (chat id, update, formattazione dei messaggi) entra nel core.**

Il confine è materializzato da due oggetti minuscoli in core/router.py:

```
AssistantRequest(text: str, user_id: str, conversation_id: str)
AssistantResponse(text: str, degraded: bool = False)
```

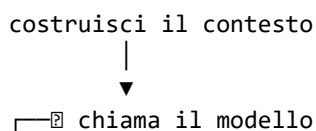
L'adapter traduce in entrambe le direzioni: da Update di Telegram a AssistantRequest, e da AssistantResponse a messaggio inviato in chat.

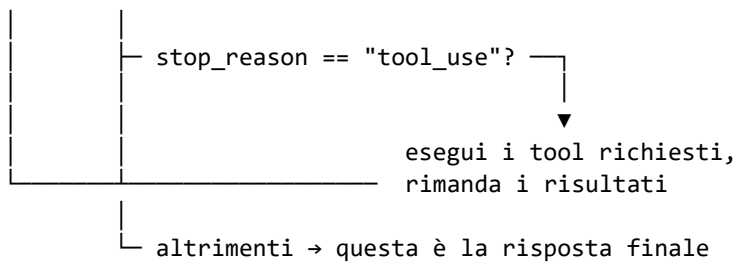
Perché conta: quando arriverà il satellite vocale sul Raspberry, la voce trascritta diventerà un AssistantRequest con lo stesso identico contratto. Il router, la memoria e il client del modello **non cambieranno di una riga**. Se invece il router leggesse update.message.chat.id, ogni nuovo canale richiederebbe di rimetterci le mani, e ogni modifica rischierebbe di rompere il canale esistente.

Il costo di questa disciplina è una decina di righe di conversione. Il beneficio è che le fasi 2, 3 e 4 della roadmap si costruiscono sopra invece che dentro.

3.4 2.4 Router agentico: il ciclo che oggi gira a vuoto

Un assistente che sa solo rispondere a parole è un chatbot. Uno che può *fare* cose deve poter chiamare degli strumenti, guardare cosa hanno restituito e decidere il passo successivo. Il protocollo dell'API Anthropic per farlo si chiama tool-use, e ha la forma di un ciclo:





Dalla v0.1 alla v0.2 la lista dei tool è rimasta **vuota**: il modello non poteva chiedere nulla e il ciclo usciva sempre al primo giro. Il codice però c'era tutto, ed era testato — proprio perché arrivasse questo momento senza doverlo riscrivere.

Dalla **v0.3** i primi tre strumenti sono registrati (paragrafo 3.3bis), e `core/router.py` non è stato toccato di una riga. Aggiungerne un altro significa scrivere una classe con quattro attributi e passarla al router:

```
class Clock:
    name = "current_time"
    description = "Restituisce l'ora corrente."
    input_schema = {"type": "object", "properties": {}}

    async def run(self, arguments: dict) -> str:
        return datetime.now().strftime("%H:%M")

router = Router(llm=llm, memory=memory, system_prompt=prompt, tools=(Clock(),))
```

`core/router.py` non si tocca. Era questo l'obiettivo.

Due protezioni sono già dentro il ciclo, perché senza di esse sarebbe fragile: un tetto al numero di round (`max_tool_iterations`, cinque), altrimenti un modello che continua a chiedere strumenti genererebbe una sequenza illimitata di chiamate a pagamento; e il contenimento degli errori dei tool, che vengono restituiti al modello come risultato con `is_error` invece di propagarsi — una skill difettosa non deve poter far cadere il turno.

3.5 2.5 Memoria dietro interfaccia

`core/memory.py` definisce un'interfaccia astratta con tre operazioni:

```
async def get_history(conversation_id) -> list[StoredMessage]
async def append(conversation_id, message) -> None
async def prune(conversation_id) -> None
```

due implementazioni: `InMemoryConversationMemory` (dizionario in RAM, perduta al riavvio, usata nei test) e `SQLiteConversationMemory` (file SQLite via `aiosqlite`, persistente tra i riavvii, attiva in produzione dalla v0.2). Entrambe condividono la stessa finestra scorrevole di `MAX_HISTORY_MESSAGES` messaggi per conversazione.

Il valore dell'interfaccia si è visto con l'introduzione della v0.2: il router non ha cambiato una riga. I test del router girano contro la memoria vera (SQLite su file temporaneo) e verificano il contratto reale indipendentemente dal backend.

Due dettagli non ovvi dell'implementazione attuale:

- **i metodi sono asincroni** anche se oggi non fanno I/O. Un database lo farà, e cambiare la firma da sincrona ad asincrona più avanti significherebbe toccare il router: esattamente ciò che l'interfaccia esiste per evitare.
- **la finestra non comincia mai con un messaggio dell'assistente**. L'API Messages rifiuta una conversazione che non parte dall'utente; se il taglio lasciasse in testa una risposta, viene scartato un messaggio in più. Senza questa regola, con un valore dispari di `MAX_HISTORY_MESSAGES` il sistema si romperebbe a caso dopo qualche scambio.

3.6 2.6 Resilienza: tre livelli

L'assistente deve fallire bene. Ci sono tre reti di sicurezza sovrapposte, ciascuna per un tipo diverso di guasto.

Livello 1 — la chiamata al modello (`core/llm.py`). Tre tentativi con attesa esponenziale: subito, dopo 1 secondo, dopo 2. Assorbe i disturbi brevi — un pacchetto perso, un 529 di sovraccarico dell'API — senza che tu te ne accorga. I retry interni del SDK sono disattivati (`max_retries=0`) perché altrimenti i tentativi reali sarebbero nove, con attese moltiplicate. Si ritenta solo ciò che ha senso ritentare: un 401 per chiave sbagliata fallisce al primo colpo, perché riprovare non la farebbe diventare giusta.

Livello 2 — il turno (`core/router.py`). Se tutti i tentativi falliscono, l'eccezione non risale: diventa una risposta di cortesia. Il processo resta vivo e il turno fallito non viene salvato in memoria — altrimenti la conversazione si riempirebbe di scuse che il modello poi cercherebbe di spiegare.

Le frasi non sono intercambiabili, perché i guasti non lo sono:

Motivo (nel log)	Cosa ricevi	Perché e' diverso
<code>model_unreachable</code>	<i>"Non riesco a contattare il cervello... riprova tra poco"</i>	riprovare ha senso
<code>quota_exhausted</code>	<i>"Ho raggiunto il limite... riprova fra circa 11 minuti"</i>	riprovare subito non ha senso, e il tempo lo dice il server
<code>empty_answer</code>	il modello ha risposto senza testo	raro, di solito un turno chiuso su un blocco tool
<code>tool_loop_ceiling</code>	troppi giri di strumenti	protegge dal ciclo infinito

Anche un guasto del **database** e' gestito qui, e non ferma la risposta: se la cronologia non si riesce a leggere il turno prosegue senza contesto, e se non si riesce a scriverla la risposta parte lo stesso (era già stata pagata in token). In entrambi i casi il log lo dice a livello ERROR.

Livello 3 — il processo (`systemd/emma.service`). Se il processo muore davvero — un bug non previsto, l'OOM killer, un riavvio della macchina — `Restart=always` lo riporta su dopo 5 secondi. Con un limite: cinque fallimenti in cinque minuti e `systemd` si ferma, perché a quel punto la causa non è transitoria (tipicamente un `.env` sbagliato) e continuare a riavviare nasconderebbe il problema invece di risolverlo.

3.7 2.7 Perché FastAPI se non c'è niente da esporre

Domanda legittima: il bot funziona in long polling, non riceve richieste HTTP. Perché un web server?

Due motivi, uno immediato e uno futuro. L'immediato è l'endpoint `/health` su `127.0.0.1:8000`: un `curl` dice se il processo è vivo, quale modello sta usando e — dalla 0.3.0 — **se sta bene davvero**, senza dover leggere i log. Il futuro è il satellite vocale sul Raspberry, che dovrà parlare con il nodo centrale via HTTP: quando arriverà, il server ci sarà già e l'avvio non andrà ripensato.

Fino alla 0.2.x l'endpoint rispondeva `"status": "ok"` in ogni circostanza, database morto compreso: un controllo che non può segnalare nulla è un controllo di vitalità con il nome sbagliato. Ora prima di rispondere legge davvero dallo store — la stessa operazione da cui dipende ogni turno, molto più economica di un `PRAGMA integrity_check` — e se non ci riesce risponde 503 con `"status": "degraded"`, così anche un controllo automatico che non sa leggere il JSON capisce lo stesso. Insieme pubblica il conteggio dei turni, quanti sono degradati, l'ultimo motivo e da quanto tempo:

```
curl -s http://127.0.0.1:8000/health | python3 -m json.tool
```

Campo	A cosa serve
<code>status</code>	ok oppure degraded: vale degraded se anche uno solo dei due sotto non va
<code>store</code> <code>telegram</code>	il database risponde? con il tipo di errore, se no listening oppure not polling — vedi sotto, è il punto cieco più insidioso
<code>version / commit</code>	quale codice sta girando davvero, non quale dovrebbe

Campo	A cosa serve
turns / degraded_turns	quanto spesso una risposta è stata di ripiego
last_degraded_reason	quale dei quattro guasti, per nome
seconds_since_degraded	da quanto: null significa mai da quando è partita

Il bot può diventare sordo senza morire. È il guasto più insidioso di tutti, e lo hai visto davvero il 31 agosto: il processo è vivo, uvicorn risponde, il database sta bene — ma il long polling di Telegram si è fermato, e nessuno può più parlarle. Dal telefono è indistinguibile da un bot spento. Fino alla 0.3.0 /health non ne sapeva niente e rispondeva ok. Ora il campo telegram dice se gli update stanno ancora arrivando, e se non arrivano l'intero stato diventa degraded con 503 — anche se tutto il resto è a posto, perché un'assistente che non ti sente non è sana.

Chi lo interroga. Dalla 0.3.0 lo fa scripts/backup.sh, cioè il job che gira comunque ogni notte alle 03:30 — ed è il posto giusto anche per un'altra ragione: ha appena copiato il database di quel servizio, quindi ha un motivo suo per volerne sapere lo stato. L'esito finisce sia nel journal sia nel MANIFEST.txt dentro l'archivio, così un backup ripristinato dice anche se il servizio stava bene nel momento in cui è stato preso:

```
git commit: 3cc4101 (v0.3.0, deployed 2026-09-01T00:17:22+02:00; from the VERSION stamp, not a checkout)
database:   emma.db (consistent snapshot, integrity verified)
service:    ok - {"status":"ok", "store":"ok", "telegram":"listening",...}
```

Un servizio spento o degradato **non fa fallire il backup**: un processo fermo è una ragione per conservare i dati, non per saltarli. Viene però scritto a chiare lettere:

```
journalctl -u emma-backup | grep WARNING | tail
```

FastAPI e il polling di Telegram condividono **un solo event loop**: uvicorn possiede il loop, e l'adapter Telegram viene avviato e fermato dal *lifespan* dell'applicazione. Questo significa che un `systemctl stop emma` chiude il polling in modo pulito invece di ucciderlo a metà di un update.

Nel REVISIONE.md trovi la discussione critica di questa scelta, insieme all'alternativa senza web server.

3.8 2.8 Il provider e il modello

EMMA supporta due backend LLM selezionabili tramite la variabile LLM_PROVIDER:

Valore	Backend	Costo
anthropic (default)	Claude via Anthropic API	a pagamento per token
groq	Llama / GPT-OSS via Groq API	tier gratuito disponibile

3.8.1 Anthropic

Il default è **Claude Sonnet 4.6** (claude-sonnet-4-6). ANTHROPIC_MODEL accetta qualunque identificativo valido — cambia la riga in .env e riavvia il servizio, senza modifiche al codice.

3.8.2 Groq (free tier)

Imposta LLM_PROVIDER=groq e fornisci una chiave gratuita da console.groq.com. Il modello di default è openai/gpt-oss-120b; puoi cambiarlo con GROQ_MODEL. I modelli disponibili dipendono dal piano dell'account — per listare quelli accessibili:

```
curl -s -H "Authorization: Bearer $GROQ_API_KEY" \
  https://api.groq.com/openai/v1/models | python3 -c \
  "import json,sys; [print(m['id']) for m in json.load(sys.stdin)['data']]"
```

Capitolo 4

Capitolo 3 — Implementazione

Questo capitolo spiega il codice file per file: a cosa serve ciascun modulo, come è fatto e perché è fatto così. Serve a te per mantenerlo e a chi vorrà contribuire per orientarsi.

4.1 3.1 La mappa

emma/	
├─ main.py	avvio: FastAPI + polling nello stesso event loop
├─ config.py	legge e valida .env
├─ adapters/	
│ └─ telegram.py	l'unico file che sa cos'è Telegram
├─ core/	
│ └─ router.py	orchestratore: contesto → modello → tool → risposta
│ └─ llm.py	client Anthropic/Groq, retry e backoff
│ └─ memory.py	interfaccia memoria + implementazioni RAM e SQLite
│ └─ tasks.py	coda dei lavori di sviluppo commissionati
├─ tools/	
│ └─ development.py	i tre strumenti con cui EMMA commissiona sviluppo
├─ prompts/	
│ └─ system_prompt.txt	la personalità di EMMA
├─ scripts/	backup, e i due script della coda di sviluppo
├─ systemd/	servizio e timer di backup
├─ tests/	suite pytest, interamente offline
├─ data/	(non in Git) database SQLite e i suoi due snapshot
└─ docs/	questa guida

La dipendenza va sempre nella stessa direzione: main.py conosce tutti, adapters/ conosce core/, **core/ non conosce nessuno** se non sé stesso.

4.2 3.2 config.py — la configurazione

Esponde due cose: la dataclass immutabile Config e la funzione load_config(), unico modo supportato per costruirla.

Il principio è **fallire subito e con chiarezza**. Ogni variabile obbligatoria mancante, ogni numero malformato, ogni file di personalità illeggibile solleva un ConfigError che nomina la variabile colpevole. Se il processo supera l'avvio, la configurazione è valida e nessun altro modulo deve più controllarla.

Dettagli che vale la pena conoscere:

- **le variabili d'ambiente reali vincono sul .env** (override=False). È ciò che permette a systemd, o a un test, di sovrascrivere una singola impostazione senza modificare file.
- **i percorsi relativi sono ancorati alla directory del progetto**, non alla working directory: SYSTEM_PROMPT_PATH=prompts/sys funziona identico che tu lanci il processo da /opt/emma o da /.

- **BACKUP_DIR e BACKUP_KEEP non sono usate dall'applicazione** — le legge `scripts/backup.sh` per conto proprio. Vengono validate qui lo stesso, così un `BACKUP_KEEP=zero` lo scopri all'avvio del servizio e non alle 3:30 di notte.
- **il file di personalità viene letto all'avvio** apposta, per trasformare un percorso sbagliato in un errore di configurazione invece che in una sorpresa al primo messaggio.

4.3 3.3 core/memory.py — la memoria

`ConversationMemory` è la classe astratta con i tre metodi visti al capitolo 2. `StoredMessage` è la coppia ruolo/testo che ci viaggia dentro.

Il modulo fornisce due implementazioni concrete.

InMemoryConversationMemory usa un dizionario di liste in RAM. È usata nei test (veloce, senza I/O) ma perde tutto al riavvio del processo.

SQLiteConversationMemory (attiva in produzione dalla v0.2) persiste i messaggi in un file SQLite via `aiosqlite`. Va aperta con `open()` all'avvio e chiusa con `close()` allo spegnimento — il lifespan di FastAPI se ne occupa. Il percorso del file si controlla con `MEMORY_DB_PATH` (default `data/emma.db`); la directory viene creata automaticamente se non esiste.

4.3.1 Auto-riparazione

All'apertura EMMA esegue `PRAGMA integrity_check` sul database. Se **fallisce**:

1. il file danneggiato viene **spostato**, mai cancellato, in `emma.db.corrotto-<data>`, insieme ai suoi file `-wal` e `-shm` (un write-ahead log vecchio non deve finire sopra il database ripristinato);
2. viene rimesso al suo posto lo snapshot più recente che supera lo stesso controllo; se anche quello è illeggibile si prova la generazione precedente;
3. se nessuno snapshot è utilizzabile, EMMA riparte con una cronologia vuota;
4. **ogni passo finisce nei log a livello ERROR**, con il percorso del file messo da parte.

Gli snapshot si scrivono con `VACUUM INTO` — che produce una copia consistente di un database in uso, cosa che una copia normale del file non garantisce — a ogni avvio riuscito e a ogni spegnimento pulito. Ne vengono tenute due generazioni (`emma.db.snapshot` e `.snapshot.prev`) e ognuna viene verificata prima di sostituire la precedente.

Il database usa `journal_mode=WAL`, molto più resistente a un'interruzione brutale (kill, OOM killer, mancanza di corrente) rispetto al journal di default.

Il recupero scatta solo su una corruzione accertata. Se EMMA non parte per un altro motivo — `.env` incompleto, dipendenza mancante, errore di codice — questo codice non viene nemmeno raggiunto, ed è voluto: ripristinare un database perché si è rotto qualcos'altro butterebbe via cronologia buona senza risolvere il guasto vero. Il ragionamento completo è nella voce 16 di `REVISIONE.md`.

Tre scelte implementative comuni a entrambe:

- **un `asyncio.Lock` protegge le sequenze leggi-modifica-scrivi.** PTB può eseguire due handler in concorrenza, e senza il lock due messaggi ravvicinati potrebbero corrompere la finestra.
- **`get_history` restituisce sempre una copia**, così chi la riceve può manipolarla senza modificare per sbaglio lo stato condiviso.
- **`prune` è idempotente**: chiamarlo due volte di fila non cambia niente. Serve perché venga invocato liberamente, senza doversi chiedere se è già stato fatto.

4.4 3.3bis core/tasks.py e tools/ — commissionare sviluppo

EMMA non può modificare il proprio codice: è il processo in esecuzione. Può però **registrare che una modifica serve**, e riferirti come sta andando. È il meccanismo introdotto nella v0.3; il ragionamento completo, incluso quello che è stato deliberatamente escluso, è la voce 17 di `REVISIONE.md`.

core/tasks.py è la coda. Un lavoro attraversa sei stadi — new, understood, implemented, committed, pushed, deployed — e a ogni passaggio si ferma in attesa di una tua risposta. Lo stage registra cosa è *fatto*; la nota ti chiede il permesso per il passo successivo.

tools/development.py contiene quattro dei sei strumenti che EMMA può chiamare:

Strumento	Quando lo usa
request_development	registri una richiesta di modifica
work_status	chiedi a che punto sono i lavori
answer_question	rispondi a una domanda in sospeso
abandon_development	vuoi togliere di mezzo un lavoro che non ti serve più

Gli altri due stanno in tools/facts/, il modulo della memoria persistente:

Strumento	Quando lo usa
remember_fact	le chiedi di ricordare qualcosa che non deve scadere
forget_fact	le chiedi di dimenticarlo

E due che parlano di lei stessa, in tools/introspection.py:

Strumento	Quando lo usa
running_version	le chiedi quale versione sta girando
list_tools	le chiedi cosa sa fare, quanti strumenti ha, o quali

E due per togliere di mezzo uno strumento, in tools/toolstate/:

Strumento	Quando lo usa
remove_tool	vuoi eliminare uno strumento — in due tempi , vedi sotto
enable_tool	vuoi riaccenderne uno spento

Perche' due tempi. Togliere uno strumento dal codice e' un lavoro di sviluppo, e non si annulla in fretta. Quindi la prima volta che lo chiedi lo strumento viene **solo disattivato**: sparisce subito dalle sue capacita' — non al prossimo riavvio, dal messaggio dopo — e lo riaccendi quando vuoi. Se glielo chiedi una seconda volta **mentre e' ancora spento**, allora registra il lavoro per toglierlo davvero dalla codebase.

Il secondo passo non e' una formalita': "gia' spento" vuol dire che ne hai fatto a meno per un po' e non ti e' mancato. La parte irreversibile non capita mai alla prima richiesta.

Due strumenti non si possono spegnere, list_tools e enable_tool: senza il primo non sapresti cosa e' spento, senza il secondo non potresti riaccenderlo — e l'unica via d'uscita sarebbe una modifica a mano sul server.

Uno strumento spento resta elencato da list_tools come (*disattivato*). Se sparisse dall'elenco non sapresti piu' cosa chiedere di riaccendere.

Perche' serve un tool per elencare i tool. Sembra assurdo che debba chiederlo: gli strumenti glieli passiamo noi a ogni richiesta. Ma le dichiarazioni arrivano al modello attraverso il campo dedicato dell'API, come *funzioni che puo' chiamare*, non come dati che puo' leggere — quindi enumerarle non e' qualcosa che sappia fare in modo affidabile su se stesso. Chiedendole quanti tool avesse, non sapeva rispondere.

list_tools riceve **la stessa tupla che riceve il router**, se stesso incluso: una lista scritta a mano sarebbe un secondo posto da aggiornare, e il primo a essere dimenticato.

Non c'è un terzo strumento per rileggere i fatti, ed è deliberato: sono già tutti davanti al modello a ogni turno, quindi uno strumento per andarli a prendere risponderebbe a una domanda di cui vede già la risposta — e ogni

dichiarazione si paga a ogni messaggio, compresi quelli in cui hai scritto solo “ciao”. Le due dichiarazioni di questo modulo costano ~303 token a turno.

Il modulo si installa e si disinstalla con **due righe in main.py**: quella che costruisce FactStore e quella che passa i tool e il fornitore di contesto al router. `core/` non sa cosa sia un fatto, esattamente come non sa cosa sia un lavoro di sviluppo.

Abbandonare non cancella. La riga resta nel database, marcata abandoned e con il motivo che hai dato: un lavoro tolto per sbaglio è ancora leggibile, e una decisione presa in un messaggio si può capire una settimana dopo. È la stessa scelta fatta per il database corrotto, che viene messo in quarantena e mai rimosso. Si possono abbandonare solo i lavori **aperti**: toglierne uno già concluso riscriverebbe la storia di ciò che è stato chiesto, non annullerebbe del lavoro. Il prompt chiede a EMMA di dirti quale lavoro sta per abbandonare e di aspettare conferma, a meno che il numero l’abbia detto tu.

Due proprietà da conoscere, perché spiegano perché è fatto così:

- **EMMA non parla mai per prima.** Nessuna riga di questo codice manda notifiche. Le domande restano nella coda e le vedi quando chiedi tu; la tua risposta torna per la stessa strada.
- **La coda vive nello stesso file SQLite della memoria.** Non è pigrizia: il controllo di integrità, gli snapshot e il backup consistente costruiti attorno a quel file coprono così anche i lavori. Un secondo database sarebbe rimasto scoperto, e in silenzio.

4.4.1 Lo stato che EMMA ha sempre davanti

DevelopmentContext non è uno strumento: è un **fornitore di contesto**. Il router lo interroga a ogni turno e accoda una riga al prompt di sistema:

Stato dei lavori di sviluppo in questo momento: 2 aperti (#1, #2). Di questi, 2 attendono una risposta dell'utente (#1, #2). Questa riga e' sempre aggiornata: se la conversazione precedente dice un numero diverso, quella e' vecchia e questa ha ragione.

Esiste per una ragione scoperta sul campo: **uno strumento viene consultato solo se il modello decide di consultarlo**, e quella decisione può andare storta. È successo — EMMA ha ripetuto parola per parola una risposta sbagliata di un quarto d’ora prima, senza andare a rileggere. La memoria persistente e gli strumenti si danneggiano a vicenda: una risposta ricavata da un tool, una volta salvata, diventa indistinguibile da un fatto.

Misurato su dieci tentativi: 6 corretti su 10 con la memoria avvelenata e nessun contesto, 10 su 10 con memoria pulita e contesto attivo. Il ragionamento completo, e le alternative scartate, sono nella voce 17.10 di REVISIONE.md.

Il punto non è il numero — che vale per *questo* modello — ma che una riga sempre presente non richiede nessuna decisione, quindi il comportamento non peggiora di nascosto il giorno in cui cambi provider.

Dall’altra parte della coda ci sono due script di shell, descritti al paragrafo 4.9: `scripts/task-queue.sh` sul server, l’unica cosa che la chiave di sviluppo è autorizzata a eseguire, e `scripts/watch-tasks.sh` sul PC, che attende senza consumare nulla.

La tabella `dev_heartbeat` registra quando una sessione di sviluppo ha guardato la coda l’ultima volta. Serve perché dietro non c’è un servizio che riparte da solo: se la sessione muore, i lavori si accumulano senza che nessuno protesti, e l’assenza di battito è l’unico modo per accorgersene. `work_status` te lo dice.

4.5 3.4 core/llm.py — il client del modello

È l’unico file che importa gli SDK dei provider (anthropic e groq). Contiene due classi con la stessa interfaccia — `AnthropicLanguageModel` e `GroqLanguageModel` — scelte in `main.py` in base a `LLM_PROVIDER`. Il router non sa quale delle due sta usando. Entrambe fanno tre cose.

Nasconde il SDK. Le risposte vengono convertite in tipi nostri — `TextBlock` e `ToolUseBlock` dentro un `LLMResponse` — così il router non dipende dagli oggetti del SDK. I blocchi di tipo sconosciuto vengono ignorati invece di sollevare un errore: un’aggiunta futura all’API non deve poter fermare l’assistente in funzione.

Applica la politica di retry. Tre tentativi, attesa 1s e 2s, timeout di 60 secondi per richiesta. Ogni tentativo fallito produce una riga di log con il tipo di errore; il successo ne produce una con `stop_reason` e i token consumati in ingresso e in uscita — è da lì che si capisce quanto costa davvero l’assistente.

Traduce il fallimento definitivo in `LLMUnavailableError`, che è ciò che il router intercetta per rispondere con cortesia. Solo gli errori transitori (problemi di connessione, 5xx) vengono ritentati: un 4xx permanente — chiave sbagliata, richiesta malformata — fallisce subito, senza bruciare tre secondi in tentativi inutili.

Un metodo merita attenzione: `to_assistant_message()`. L’API Messages è *stateless*, quindi per continuare un turno agentico bisogna rimandare la risposta precedente del modello parola per parola, blocchi di tool compresi. Quel metodo la ricostruisce nel formato giusto.

4.6 3.5 core/router.py — l’orchestratore

Il cuore. Contiene gli oggetti di confine (`AssistantRequest`, `AssistantResponse`), il protocollo Tool e la classe Router.

`handle()` esegue un turno completo e **non solleva mai** eccezioni dovute al modello o a un tool: qualunque guasto diventa una risposta degradata ma educata. I tre messaggi di fallback (modello irraggiungibile, risposta vuota, troppi passaggi) sono costanti in cima al file — sono le uniche stringhe italiane del codice, perché sono le uniche che leggi tu e non un programmatore.

`_run_agentic_loop()` è il ciclo tool-use. `_execute_tool()` esegue un singolo strumento e ne cattura le eccezioni, restituendole al modello come risultato d’errore.

Una regola di comportamento che vale la pena conoscere: **i turni degradati non vengono salvati in memoria**. Se il modello era irraggiungibile, la tua domanda e la risposta di cortesia spariscono, così il messaggio successivo riparte da una cronologia pulita invece che da una scusa.

4.7 3.6 adapters/telegram.py — il canale

Costruisce l’Application di python-telegram-bot, registra un solo handler per i messaggi di testo e un gestore di errori che logga qualunque eccezione sfugga, tenendo il bot in piedi.

Punti da conoscere:

- **la whitelist è un controllo esplicito nell’handler**, non un filtro di PTB, perché così un tentativo da parte di uno sconosciuto lascia una riga WARNING nei log. Se qualcuno trova il tuo bot, te ne accorgi.
- **drop_pending_updates=True all’avvio**. Dopo un riavvio ricevi un assistente vivo, non una raffica di risposte a domande di tre ore prima.
- **le risposte lunghe vengono spezzate**. Telegram rifiuta i messaggi oltre 4096 caratteri; il taglio preferisce un a capo vicino al limite, per non spezzare righe e paragrafi a metà.
- **l’indicatore “sta scrivendo”** viene inviato prima di chiamare il modello, così i due secondi di attesa non sembrano un silenzio.
- **i comandi (/start e simili) sono ignorati** nella v1. È una semplificazione voluta: l’unica interazione è scrivere in linguaggio naturale.

4.8 3.7 main.py — l’avvio

È il *composition root*: l’unico punto in cui si scelgono le classi concrete.

```
llm      = AnthropicLanguageModel(...)          # o GroqLanguageModel, secondo LLM_PROVIDER
memory   = SQLiteConversationMemory(...)        # persistente dalla v0.2
router   = Router(llm, memory, prompt, tools=()) # ← qui si registrano le skill
telegram = TelegramAdapter(token, user_id, router)
```

Sostituire un componente è una riga in questo file. È il posto giusto in cui guardare per capire come è montato il sistema.

Il *lifespan* di FastAPI apre il database, avvia il polling all’avvio del server e allo spegnimento fa il percorso inverso: ferma il polling, chiude il pool di connessioni HTTP verso il provider e chiude il database. Il logging è configurato

una volta sola su stdout, nel formato `timestamp | LIVELLO | logger | messaggio`, con i logger delle librerie HTTP silenziati: sotto long polling emetterebbero una riga ogni pochi secondi senza dire nulla.

`main()` restituisce 2 se la configurazione non è valida, e logga l'errore senza traceback: un `.env` sbagliato è un errore d'uso, e trenta righe di stack nasconderebbero l'unica che conta.

4.9 3.8 tests/ — la suite

Quarantatré test, tutti offline. Il modello è sostituito da `ScriptedModel`, un finto client che restituisce risposte preparate e registra cosa gli è stato chiesto: implementa la stessa interfaccia del client vero, quindi verifica il contratto reale.

Come sono distribuiti:

File	Test	Cosa copre
<code>test_router.py</code>	12	turno semplice, cronologia, isolamento fra conversazioni, ciclo tool completo con formato dei <code>tool_result</code> , tool sconosciuto, tool che esplode, tetto ai round, modello irraggiungibile, risposta vuota, turno fallito che non inquina il successivo
<code>test_memory_sqlite.py</code>	10	come sopra, più la persistenza attraverso chiusura e riapertura del database
<code>test_memory.py</code>	9	finestra scorrevole, regola del primo messaggio utente, idempotenza di prune, copia difensiva
<code>test_llm.py</code>	6	distinzione fra errori transitori (da ritentare) e permanenti (da propagare subito)
<code>test_telegram.py</code>	6	<code>_split_message</code> , incluse le righe vuote a cavallo di un taglio

Non puntano alla copertura totale: puntano ai punti in cui una regressione sarebbe silenziosa.

```
pytest          # nella directory del progetto, con il virtualenv attivo
```

Capitolo 5

Capitolo 4 — Deploy

Da qui in avanti si lavora sul server, con l'ambiente preparato al capitolo 1.

5.1 4.1 Il repository su GitHub

Il progetto nasce come repository Git. Sul PC di sviluppo Windows, dalla cartella del progetto:

```
git init
git add .
git commit -m "initial commit: EMMA v0.1.0, text-only assistant"
git tag -a v0.1.0 -m "v0.1.0 - first working release"
```

Crea poi un repository **privato** su GitHub (lo renderai pubblico tu al momento della release) e collegalo:

```
git remote add origin https://github.com/<tuo-account>/emma.git
git branch -M main
git push -u origin main --tags
```

Verifica. Su GitHub devi vedere i file e **non** devi vedere `.env`. Se lo vedi, fermati: il file è stato committato, la chiave è compromessa e va revocata subito su `console.anthropic.com` prima di qualunque altra cosa.

5.2 4.2 Scaricare il codice sul server

```
sudo -u emma git clone https://github.com/<tuo-account>/emma.git /opt/emma
cd /opt/emma
```

Se la directory `/opt/emma` esiste già ed è vuota (l'abbiamo creata al capitolo 1), `git clone` la usa senza problemi. Se protesta perché non è vuota, clona in `/tmp` e sposta il contenuto.

Con repository privato, `git clone` chiede le credenziali: usa un *personal access token* GitHub al posto della password, oppure una chiave SSH di deploy. Il token va salvato con `git config --global credential.helper store` **solo** se accetti che finisca in chiaro in `~/ .git-credentials` dell'utente `emma`.

5.3 4.3 L'ambiente virtuale

```
sudo -u emma python3 -m venv /opt/emma/.venv
sudo -u emma /opt/emma/.venv/bin/pip install --upgrade pip
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
```

Il `virtualenv` isola le dipendenze di EMMA dal Python di sistema: nessun rischio di rompere strumenti di Ubuntu che usano Python, e nessun bisogno di `--break-system-packages`.

Verifica.

```
sudo -u emma /opt/emma/.venv/bin/pip list | grep -Ei "anthropic|groq|aiosqlite|telegram|fastapi|uvicorn|
```

Devi vedere tutte le librerie con le versioni esatte scritte in requirements.txt.

5.4 4.4 Il file .env

```
sudo -u emma cp /opt/emma/.env.example /opt/emma/.env
sudo -u emma chmod 600 /opt/emma/.env
sudo -u emma nano /opt/emma/.env
```

600 significa: solo l'utente emma può leggerlo e scriverlo. Nessun altro utente della macchina, nemmeno il tuo, può vedere la chiave API senza sudo.

Compila i valori obbligatori con le credenziali del paragrafo 1.9.

Con Anthropic (default):

```
LLM_PROVIDER=anthropic
ANTHROPIC_API_KEY=sk-ant-...           # la tua chiave
TELEGRAM_BOT_TOKEN=123456789:AAH...   # il token di BotFather
TELEGRAM_ALLOWED_USER_ID=123456789    # il tuo ID numerico
```

Con Groq (free tier):

```
LLM_PROVIDER=groq
GROQ_API_KEY=gsk-...                  # chiave da console.groq.com
TELEGRAM_BOT_TOKEN=123456789:AAH...
TELEGRAM_ALLOWED_USER_ID=123456789
```

E controlla gli opzionali, che hanno già default sensati:

Variabile	Default	Significato
LLM_PROVIDER	anthropic	backend LLM: anthropic o groq
ANTHROPIC_MODEL	claude-sonnet-4-6	modello Anthropic
GROQ_MODEL	openai/gpt-oss-120b	modello Groq
MAX_HISTORY_MESSAGES	20	messaggi tenuti nella finestra di contesto
MEMORY_DB_PATH	data/emma.db	file SQLite della cronologia; creato al primo avvio
SYSTEM_PROMPT_PATH	prompts/system_prompt.txt	il file con la personalità
BACKUP_DIR	/mnt/backup/emma	dove finiscono gli archivi
BACKUP_KEEP	14	quanti archivi conservare

Se al capitolo 1.7 hai montato il disco su un percorso diverso, correggi BACKUP_DIR adesso.

Verifica.

```
ls -l /opt/emma/.env
# -rw----- 1 emma emma ... /opt/emma/.env
git -C /opt/emma status --short
# non deve comparire .env
```

5.5 4.5 Primo avvio a mano

Prima di installare il servizio, prova che tutto funzioni in primo piano, dove i messaggi d'errore si leggono subito:

```
cd /opt/emma
sudo -u emma /opt/emma/.venv/bin/python main.py
```

Devi vedere qualcosa come:

```
2026-08-31T14:02:10+0200 | INFO      | emma | starting emma (provider=anthropic, model=claude-sonnet-4-6, history=20 messages, db=data/emma.db)
2026-08-31T14:02:11+0200 | INFO      | adapters.telegram | telegram adapter started (long polling)
INFO:      Unicorn running on http://127.0.0.1:8000
```

Al primo avvio il file del database viene creato automaticamente dentro data/.

La directory data/ va creata prima di installare il servizio, al paragrafo 4.6: emma.service la dichiara in ReadWritePaths=, e systemd **rifiuta di avviare** una unit il cui ReadWritePaths punta a una directory che non esiste. Se stai provando a mano come qui sopra, EMMA la crea da sé.

Ora la prova vera: prendi il telefono e scrivi al tuo bot. Entro un paio di secondi devi ricevere una risposta, e sul terminale devono comparire le righe incoming message from chat_id=... e answered chat_id=...

Se non succede nulla, salta al paragrafo 6.7: i tre motivi più comuni sono un token sbagliato, un TELEGRAM_ALLOWED_USER_ID che non è il tuo, e l'aver scritto a un bot diverso da quello del token.

Ferma il processo con Ctrl+C.

5.6 4.6 Il servizio systemd

Prima la directory del database, che deve esistere **prima** che la unit parta:

```
sudo -u emma mkdir -p /opt/emma/data
sudo chmod 700 /opt/emma/data
```

700 perché contiene le tue conversazioni. Poi il servizio:

```
sudo cp /opt/emma/systemd/emma.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now emma.service
```

enable --now fa due cose insieme: avvia il servizio adesso e lo configura per partire da solo a ogni avvio della macchina.

Se hai installato EMMA in un percorso diverso da /opt/emma, prima di copiare il file modifica le righe marcate # PATH: e la coppia User=/Group=.

Perché la directory prima. La unit è blindata con ProtectSystem=strict, che rende l'intero filesystem in sola lettura per il servizio; l'unica eccezione è ReadWritePaths=/opt/emma/data, che è ciò che permette a EMMA di scrivere la cronologia senza poter riscrivere il proprio codice. Systemd però **rifiuta di avviare** una unit il cui ReadWritePaths non esiste, con un errore poco parlante (Failed to set up mount namespaces). Se sposti il database con MEMORY_DB_PATH, aggiorna quella riga nella unit e la gemella in emma-backup.service.

Verifica.

```
systemctl status emma.service
```

Devi leggere Active: active (running). Poi:

```
curl -s http://127.0.0.1:8000/health
# {"status":"ok","store":"ok","model":"claude-sonnet-4-6","provider":"anthropic",
#  "version":"0.3.0","commit":"a1b2c3d","uptime_seconds":12.4,
#  "turns":0,"degraded_turns":0,"last_degraded_reason":null,
#  "seconds_since_degraded":null}

journalctl -u emma -n 30 --no-pager
```

E di nuovo la prova che conta: scrivi al bot dal telefono e verifica di ricevere risposta.

Verifica del riavvio automatico (è un criterio di accettazione, vale la pena provarlo davvero):

```
sudo systemctl kill -s SIGKILL emma.service    # simula un crash brutale
sleep 8
systemctl status emma.service                  # deve essere di nuovo running
```

Nei log vedrai la ripartenza. Se dopo dieci secondi il servizio non è tornato su, qualcosa nella unit non va: controlla `journalctl -u emma -n 50`.

5.7 4.7 Il timer di backup

```
sudo cp /opt/emma/systemd/emma-backup.service /etc/systemd/system/
sudo cp /opt/emma/systemd/emma-backup.timer /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now emma-backup.timer
```

La unit copre già entrambe le destinazioni possibili (`/mnt/backup` se c'è, `/var/backups` altrimenti) e crea da sé `/var/backups/emma` con i permessi giusti, così il ripiego funziona anche su una macchina appena installata. Se invece hai impostato un `BACKUP_DIR` che non sta in nessuna delle due, aggiungi quella directory a `ReadWritePaths=` in `emma-backup.service` (riga marcata `# PATH:`) prima di copiarlo: con `ProtectSystem=strict` il filesystem è in sola lettura per il servizio, e senza quella riga il backup fallisce con un permesso negato.

Se lanci `scripts/backup.sh` a mano come utente `emma` prima che il timer sia mai partito, la directory di ripiego potrebbe non esistere ancora: `sudo install -d -o emma -g emma -m 700 /var/backups/emma` la crea. Passando dal servizio (`systemctl start emma-backup.service`) non serve.

Fai subito una prova a mano, senza aspettare le 3:30:

```
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 30 --no-pager
```

Nel log, la riga `destination:` dice dove è finito l'archivio e perché:

```
destination: /mnt/backup/emma [separate disk]
destination: /var/backups/emma [system disk, fallback - no separate disk available]
```

Guarda lì e poi elenca quella directory:

```
ls -lh /mnt/backup/emma/    # oppure /var/backups/emma/
```

Verifica. Deve esserci un file `emma-AAAAMMGG-HHMMSS.tar.gz` con permessi `-rw-----`. Controlla anche il contenuto:

```
tar -tzf /mnt/backup/emma/emma-*.tar.gz | head -20
tar -xzOf /mnt/backup/emma/emma-*.tar.gz MANIFEST.txt
```

Devono comparire i file del progetto, il `.env` e il manifesto con data e commit Git di provenienza.

```
systemctl list-timers emma-backup.timer
```

Ti dice quando scatterà la prossima esecuzione.

5.8 4.8 Riepilogo: cosa c'è ora sulla macchina

Percorso	Cosa contiene	Permessi
<code>/opt/emma</code>	codice, virtualenv, prompt	750 emma:emma
<code>/opt/emma/.env</code>	chiave API e token	600 emma:emma
<code>/opt/emma/data/emma.db</code>	cronologia delle conversazioni	emma:emma
<code>/opt/emma/data/emma.db.snapshots</code>	copie per il recupero	emma:emma
<code>/mnt/backup/emma</code>	archivi datati	700 emma:emma
<code>/etc/systemd/system/emma.service</code>	il servizio	root

Percorso	Cosa contiene	Permessi
/etc/systemd/system/emma-backup.{service,timer}	il backup	root

Nessuna porta in ascolto verso l'esterno; una sola regola nel firewall, per SSH.

5.9 4.9 La coda di sviluppo

Serve solo se vuoi commissionare sviluppo a EMMA (paragrafo 5.6). Se salti questo paragrafo, EMMA registra comunque le richieste: semplicemente nessuno le raccoglie.

Il meccanismo è una sessione di sviluppo, sul PC dove c'è il repository, che legge la coda sul server. Per farlo le serve una chiave SSH — e qui c'è una scelta che vale la pena fare bene.

5.9.1 4.9.1 Perché una chiave dedicata

Quella sessione interroga il server **di continuo e senza che nessuno guardi**. Darle la chiave di amministrazione significherebbe mettere la credenziale più potente della macchina nell'unico percorso non sorvegliato.

La chiave dedicata è invece vincolata a un solo script, e la restrizione la applica sshd, non una buona intenzione: chi presenta quella chiave esegue scripts/task-queue.sh e nient'altro, qualunque comando chieda. Se venisse rubata, il danno massimo è scrivere sciocchezze nella coda dei lavori — che poi leggi tu — non toccare il sistema.

La chiave di amministrazione resta quella che hai e si usa solo per il deploy, che è comunque dietro una tua conferma.

5.9.2 4.9.2 Sul PC di sviluppo

```
ssh-keygen -t ed25519 -f ~/.ssh/emma_queue -C "emma task queue" -N ""
cat ~/.ssh/emma_queue.pub
```

Poi aggiungi la destinazione a ~/.ssh/config, che **non** sta nel repository:

```
Host emma-queue
    HostName <il-tuo-server>
    User emma
    IdentityFile ~/.ssh/emma_queue
    IdentitiesOnly yes
```

IdentitiesOnly yes evita che SSH provi prima le altre chiavi che hai in giro, compresa quella di amministrazione.

5.9.3 4.9.3 Sul server

Aggiungi la chiave pubblica appena creata a /opt/emma/.ssh/authorized_keys, preceduta dalle restrizioni, **tutto su una riga sola**:

```
sudo -u emma tee -a /opt/emma/.ssh/authorized_keys <<'EOF'
command="/opt/emma/scripts/task-queue.sh",no-pty,no-port-forwarding,no-agent-forwarding,no-X11-forwarding
EOF
sudo -u emma chmod 600 /opt/emma/.ssh/authorized_keys
sudo -u emma chmod 750 /opt/emma/scripts/task-queue.sh
```

Verifica. Dal PC di sviluppo:

```
ssh emma-queue touch           # deve rispondere: ok
ssh emma-queue list           # La coda, in JSON
ssh emma-queue whoami         # deve essere RIFIUTATO
```

L'ultimo comando è quello che conta: se ti risponde con un nome utente invece di refused, la riga command= non è stata applicata e la restrizione non esiste. Ricontrolla che sia tutta su una riga e prima della chiave.

5.9.4 4.9.4 Le operazioni ammesse

scripts/task-queue.sh accetta soltanto questi verbi, e rifiuta ogni altra cosa. Non accetta mai SQL: costruisce lui le query, e ogni valore che ci finisce dentro è un intero verificato oppure una stringa con gli apici raddoppiati.

Comando	Cosa fa
list	i lavori che aspettano lo sviluppatore, in JSON
list-all	tutti i lavori, compresi quelli chiusi e abbandonati
show <n>	un lavoro
touch	registra che la sessione è viva
create "<descrizione>"	apre un lavoro trovato lavorando al codice
advance <n> <stadio> "<nota>"	avanza e pone la domanda del checkpoint
finish <n> "<nota>"	chiude un lavoro deployato
abandon <n> "<nota>"	lo lascia perdere

Ogni comando aggiorna anche il battito: una sessione che sta lavorando è viva, e sarebbe assurdo che risultasse morta perché non ha chiamato touch.

create esiste per un caso solo: un difetto scoperto **mentre** si lavora al codice, che altrimenti resterebbe nella memoria di chi l’ha visto. Non sposta il controllo — un lavoro aperto così si ferma comunque al primo checkpoint e ti chiede “*procedo?*” prima che venga costruito qualcosa. I lavori che nascono da te continuano ad arrivare da EMMA (paragrafo 5.6).

5.9.5 4.9.6 I tre momenti in cui puoi accorgerti di un lavoro

Dietro la coda non c’è un servizio: se nessuno guarda, i lavori si accumulano. Fino alla 0.3.0 esisteva un solo hook, SessionStart, che guardava **all’apertura della sessione e mai più** — così un lavoro commissionato mentre la sessione era già aperta non veniva notato da nessuno. È successo davvero il 1 settembre 2026.

I momenti sono tre, e servono meccanismi diversi:

Quando arriva il lavoro	Cosa lo nota
Prima che la sessione si apra	hook SessionStart → queue-brief.sh
A sessione aperta, e poi scrivi	hook UserPromptSubmit → queue-brief.sh
A sessione aperta, e non scrivi	hook Stop → watch-tasks.sh in asyncRewake

In .claude/settings.local.json del posto da cui lavori:

```
{
  "hooks": {
    "SessionStart": [
      { "hooks": [
        { "type": "command",
          "command": "bash '<percorso>/emma/scripts/queue-brief.sh' SessionStart 10",
          "timeout": 20 },
        { "type": "command",
          "command": "EMMA_WAKE_ON_WORK=1 POLL_SECONDS=120 bash '<percorso>/emma/scripts/watch-tasks.sh'",
          "asyncRewake": true, "timeout": 21600 }
      ] }
    ],
    "UserPromptSubmit": [
      { "hooks": [ { "type": "command",
        "command": "bash '<percorso>/emma/scripts/queue-brief.sh' UserPromptSubmit 4",
        "timeout": 15 } ] }
    ],
    "Stop": [
      { "hooks": [ { "type": "command",
```

```

    "command": "EMMA_WAKE_ON_WORK=1 POLL_SECONDS=120 bash '<percorso>/emma/scripts/watch-tasks.sh'",
    "asyncRewake": true, "timeout": 21600 } ] }
}
}

```

Il nome dell'evento è un argomento perché Claude Code **scarta** l'output il cui hookEventName non corrisponde all'hook che l'ha prodotto. Il timeout di connessione è il secondo: all'apertura dieci secondi spesi per sapere sono gratis, su ogni messaggio sono dieci secondi in cui aspetti.

Riporta **solo il numero**, non il testo delle richieste: leggerle costerebbe contesto in ogni sessione, comprese quelle in cui non verranno toccate. Se la coda è vuota, o il server è irraggiungibile e non c'è cache, non stampa niente ed esce con successo — una sessione non deve fallire l'avvio, né un messaggio restare bloccato, perché una macchina è spenta.

La cache locale, e perché è solo una rete di sicurezza. Ogni interrogazione riuscita scrive lo stato in `~/.claude/emma-queue-state`. Se in seguito il server non risponde, quel numero viene riportato dicendo esplicitamente che è vecchio e di quanto: *“(il server non risponde; dato di 4 minuti fa)”*.

L'ordine è deliberatamente l'opposto di quello che sembra ovvio: **prima il server, la cache solo se fallisce**. Leggere prima la cache renderebbe l'hook istantaneo, ma una cache vecchia anche di pochi minuti può non contenere il lavoro appena inserito — cioè esattamente il difetto che tutto questo esiste per chiudere. La freschezza è la funzione; la cache compra robustezza senza spenderne.

A riscriverla sono gli hook stessi: ogni apertura di sessione e ogni tuo messaggio la aggiornano, perché ognuno di quei momenti interroga già il server. Non serve nient'altro.

Un'attività pianificata: provata e disattivata. Il 1 settembre 2026 ne è stata registrata una che rinfrescava la cache ogni 5 minuti anche a sessione chiusa (`scripts/queue-brief.sh --refresh`). Ha funzionato, e va comunque tolta di mezzo, per due ragioni che si sono viste solo all'uso.

Si vedeva. Eseguendo Git Bash come attività Interactive, faceva lampeggiare una finestra di terminale sullo schermo ogni cinque minuti. L'opzione `-Hidden` di `New-ScheduledTaskSettingsSet` non serve a questo: nasconde l'attività nell'elenco dell'Utilità di pianificazione, non la finestra. Per non vederla servirebbe farla girare nella sessione 0 (`-LogonType S4U`), il che apre la domanda se le chiavi SSH funzionino ancora da lì.

E comprava pochissimo. La cache è già aggiornata a ogni messaggio e a ogni apertura; l'attività aggiungeva solo aggiornamenti mentre nessuna sessione è aperta — cioè quando non c'è nessuno da avvisare. L'unico guadagno reale era che una sessione aperta *dopo* un guasto del server trovasse un numero un po' meno vecchio.

Un fastidio permanente per un guadagno marginale è uno scambio sbagliato. L'attività resta registrata ma **disattivata**; si toglie del tutto con `Unregister-ScheduledTask -TaskName 'EMMA queue cache'`.

5.9.6 4.9.5 L'attesa che non costa

`scripts/watch-tasks.sh`, sul PC di sviluppo, interroga la coda e **termina appena c'è qualcosa**. È tutto qui il trucco: ad aspettare è uno script di shell, che non costa niente, e la sessione che invece costa si sveglia solo quando c'è davvero lavoro. Una giornata senza richieste è una giornata di shell che dorme.

```

scripts/watch-tasks.sh           # ogni 5 minuti, per 6 ore
POLL_SECONDS=60 scripts/watch-tasks.sh  # più reattivo

```

Dalla 0.3.0 non va più avviato a mano. Gli hook `SessionStart` e `Stop` lo lanciano con `asyncRewake`, che è il meccanismo con cui Claude Code sveglia il modello quando un comando in background **esce con codice 2**. Il `Stop` lo riarma dopo ogni turno, quindi resta attivo per tutta la durata della sessione.

Due dettagli senza i quali si romperebbe:

- **In modo hook i codici di uscita sono invertiti.** Normalmente 2 significa *“ho rinunciato, non c'è niente”*; con `asyncRewake` significherebbe *“svegliati”*, cioè il contrario esatto. Con `EMMA_WAKE_ON_WORK=1`, 2 vuol dire

lavoro nuovo e **tutto il resto esce 0 in silenzio** — una sveglia afferma che qualcosa aspetta, e non va spesa su un guasto di rete.

- **Ricorda cosa ha già annunciato.** Senza memoria, il Stop lo riavvierebbe, troverebbe lo stesso lavoro ancora in coda, e sveglierebbe di nuovo — per sempre, se quel lavoro aspetta una tua risposta. Un lucchetto rende inoltre il riarmo idempotente: chiedere un guardiano mentre uno sta già guardando non fa niente.

Resta comunque legato alla sessione. Muore con essa, e se un lavoro arriva nei pochi secondi fra la sveglia e il riarmo lo trova al giro successivo. Renderlo davvero garantito vorrebbe dire un servizio che sopravvive alla sessione: l'infrastruttura che questo progetto ha scelto di non avere.

La parte affidabile è l'hook del paragrafo 4.9.6, che scatta a ogni apertura di sessione e non richiede nessun processo attivo. Insieme danno: **apri e sai subito se c'è lavoro; mentre lavori, se il guardiano è vivo ti sveglia.** Quello che non si può promettere è "commissiono di notte e lo trovo fatto".

Capitolo 6

Capitolo 5 — Utilizzo

6.1 5.1 Il giorno per giorno

Apri Telegram, scrivi al bot, ricevi la risposta. È tutto.

EMMA ricorda la conversazione: puoi fare domande di seguito senza ripetere il contesto (“e domani?” dopo aver chiesto del meteo funziona). La memoria copre gli ultimi MAX_HISTORY_MESSAGES messaggi — con il default di 20, circa dieci scambi — e **dalla v0.2 sopravvive ai riavvii**: dopo un `systemctl restart` o un riavvio della macchina la conversazione riparte da dove l’avevi lasciata.

Ma quella finestra dimentica per anzianità, e l’anzianità è il criterio sbagliato per certe cose: *“mia figlia si chiama Sara”* scade esattamente alla stessa velocità di *“che ore sono”*. Dopo una decina di scambi non è “sbiadito”, è cancellato dal database.

Per questo esistono i **fatti**. Se le dici *“ricorda che il wifi di casa è X”*, EMMA lo registra a parte e non scade mai: lo saprà fra un mese, dopo qualsiasi riavvio. Funziona **solo se glielo chiedi** — *“ricorda che”, “segnati che”, “non dimenticare che”* — perché decidere da sola cosa merita di essere ricordato è la stessa classe di rischio del riassunto automatico: sbaglia in modo plausibile, e nessuno pensa a verificarlo.

Per farle dimenticare qualcosa basta chiederglielo. Il fatto non viene distrutto, viene solo messo da parte: come per un lavoro abbandonato e per un database corrotto, una decisione che si può rileggere è meno definitiva di una che non si può.

Costa qualcosa, ed è giusto saperlo. I fatti vengono messi davanti al modello a ogni singolo messaggio, quindi si pagano ogni volta. Misurato sul traffico vero: senza nessun fatto uno scambio passa da ~2.360 a ~2.660 token, con trenta fatti a ~3.100. Sul tetto gratuito di Groq da 200.000 token al giorno significa passare da ~84 scambi a ~64. Il massimo è **50 fatti**.

Tempi di risposta tipici: uno o due secondi. L’indicatore “sta scrivendo” compare subito, così sai che il messaggio è arrivato.

6.2 5.2 Cosa aspettarsi

EMMA nella v1 è una conversazione con un modello linguistico, con una personalità definita in `prompts/system_prompt.txt`: risposte brevi, italiano, tono diretto, nessun preambolo. Va bene per ragionare su un problema, farsi spiegare qualcosa, buttare giù un testo, riordinare le idee.

Non ha accesso a Internet in tempo reale, ai tuoi file, al calendario, alla casa. Se le chiedi il meteo di adesso ti dirà che non può saperlo — e questo è il comportamento voluto: meglio un “non lo so” onesto di un’informazione inventata.

6.3 5.3 Personalizzare il carattere

Il file `prompts/system_prompt.txt` è la personalità, in italiano, in chiaro. Modificalo quando vuoi:

```
sudo -u emma nano /opt/emma/prompts/system_prompt.txt
sudo systemctl restart emma.service
```

Il prompt viene letto all'avvio, quindi il **riavvio serve**. Qualche consiglio: descrivi il comportamento, non l'identità ("rispondi in due frasi" funziona meglio di "sii conciso"); dichiara esplicitamente cosa non può fare, così eviti che inventi; e tieni il file corto, perché viene inviato a ogni singolo messaggio e quindi lo paghi ogni volta.

Se modifichi questo file **committalo**: fa parte del progetto e va versionato come il codice.

6.4 5.4 Quanto costa

Dipende dal provider. Con LLM_PROVIDER=groq sul piano gratuito **non costa nulla**, entro i limiti di richieste al minuto dell'account: è l'opzione da preferire se l'obiettivo è tenere la spesa a zero.

Con Anthropic si paga a token. Ogni messaggio consuma token in ingresso (il prompt di sistema, la finestra di conversazione, la tua domanda) e in uscita (la risposta). Con Sonnet e un uso personale si parla di pochi euro al mese, ma dipende da quanto scrivi.

I numeri veri sono nei log:

```
journalctl -u emma | grep -E "(anthropic|groq) call ok" | tail -20
# ... anthropic call ok (attempt 1): stop_reason=end_turn in=412 out=87
```

in e out sono i token consumati. Il consuntivo ufficiale è nella dashboard di Anthropic; il consiglio pratico è impostare lì un limite di spesa mensile.

Il parametro che sposta di più il costo è MAX_HISTORY_MESSAGES: raddoppiarlo raddoppia all'incirca i token in ingresso di ogni messaggio, perché l'intera finestra viene rimandata ogni volta.

6.5 5.5 Limiti noti della versione 1

Sono limiti dichiarati, non difetti. Ognuno ha già la sua fase nella roadmap.

- **Quasi nessuno strumento.** Dalla v0.3 EMMA sa commissionare il proprio sviluppo (paragrafo 5.6), ma non ha ancora meteo, calendario, note, luci o ricerche. Quelli si chiedono, appunto, commissionandoli.
- **Nessuna voce.** Solo testo, solo Telegram. → v0.4, satellite Raspberry.
- **Solo testo in ingresso.** Foto, audio e documenti inviati al bot vengono ignorati senza risposta.
- **I comandi Telegram non fanno nulla.** /start e simili sono ignorati: scrivi normalmente.
- **Un solo utente.** Chiunque altro viene ignorato in silenzio, per progetto.
- **Nessuna cancellazione della conversazione dal telefono.** Dalla v0.2 la memoria è persistente, quindi un riavvio non basta più: per ripartire da zero cancella il file del database e riavvia il servizio (paragrafo 5.7).

6.6 5.6 Commissionare uno sviluppo

Se EMMA non sa fare una cosa, puoi chiedere che impari a farla. Non la implementa lei — registra la richiesta, uno sviluppatore la prende in carico, e alla fine EMMA riparte con la capacità in più.

Per registrare una richiesta ci sono due modi:

sviluppo: quando ti chiedo il meteo, dimmi che non hai internet

Con il prefisso sviluppo: la registra subito, senza discutere. Altrimenti puoi dirlo normalmente e sarà lei a proporlo:

tu: vorrei che ricordassi i miei appuntamenti **EMMA:** Questa richiede una modifica al mio codice. La registro come lavoro? **tu:** sì **EMMA:** Registrata come lavoro #3.

Non registra mai niente di sua iniziativa: o sei esplicito tu, o te lo chiede.

Per sapere a che punto sono:

tu: a che punto sono i lavori? **EMMA:** #3 [implementato e testato, in attesa di essere committato] vorrei che ricordassi i miei appuntamenti — **DOMANDA:** 53 test verdi. Committo? #4 [in attesa che lo sviluppatore lo prenda in carico] ...

Per rispondere a una domanda:

tu: sì al 3, committa pure **EMMA:** Risposta registrata sul lavoro #3.

Ogni lavoro si ferma quattro volte in attesa di un tuo sì: prima di implementare, prima di committare, prima di pubblicare su GitHub e prima del deploy. È voluto — il momento in cui una richiesta capita male costa poco è prima che diventi codice pubblicato.

EMMA non ti scrive mai per prima. Le domande restano lì finché non chiedi tu. Se non chiedi per un giorno, il lavoro si ferma al primo cancello e aspetta: nulla si perde, ma nulla prosegue.

Se EMMA dice che la sessione di sviluppo non è attiva, come qui:

NOTA: l'ultimo contatto con la sessione di sviluppo risale a 2 giorni fa.
Probabilmente non è attiva.

...significa esattamente quello: dietro non c'è un servizio che riparte da solo, ma una sessione che qualcuno ha lasciato aperta sul PC di sviluppo. Se è chiusa, le richieste si accumulano e nessuno le raccoglie. Riaprila.

6.7 5.7 Azzerare la memoria

La cronologia sta in un file SQLite, per default `/opt/emma/data/emma.db`, affiancato da due snapshot. Per ripartire da zero vanno via tutti e tre:

```
sudo systemctl stop emma.service
sudo -u emma rm -f /opt/emma/data/emma.db /opt/emma/data/emma.db.snapshot*
sudo systemctl start emma.service
```

Il database viene ricreato vuoto al primo messaggio.

Cancellare solo emma.db non basta. Il file sparirebbe, ma la cronologia resterebbe negli snapshot — e se un giorno il nuovo database si corrompesse, EMMA ripristinerebbe le conversazioni che credevi di aver cancellato. L'asterisco nel comando sopra serve esattamente a questo.

Se vuoi conservare la cronologia prima di cancellarla, copiala altrove: è un file SQLite normale, leggibile con `sqlite3 emma.db "SELECT * FROM messages;"`. Tienilo dove tieni i backup — contiene le tue conversazioni.

Capitolo 7

Capitolo 6 — Manutenzione

7.1 6.1 Leggere i log

Tutto finisce nel journal di systemd. I comandi che userai davvero:

```
journalctl -u emma -f           # in diretta (Ctrl+C per uscire)
journalctl -u emma -n 100 --no-pager # ultime 100 righe
journalctl -u emma --since "1 hour ago" # ultima ora
journalctl -u emma --since today -p err # solo errori di oggi
journalctl -u emma -u emma-backup --since "2 days ago" # servizio e backup insieme
```

Il formato di ogni riga è timestamp | LIVELLO | modulo | messaggio. Cosa significano le righe che vedrai più spesso:

Riga	Significato
starting emma (provider=..., model=..., history=..., db=...)	avvio, con la configurazione in uso
telegram adapter started (long polling)	il bot è connesso e in ascolto
incoming message from chat_id=...	è arrivato un tuo messaggio
anthropic call ok (attempt 1): ... in=N out=M	risposta ottenuta, token consumati (con Groq: groq call ok)
answered chat_id=... (degraded=False)	risposta inviata
ignored message from user_id=... (not in whitelist)	qualcun altro ha scritto al bot
anthropic call failed (attempt 1/3)	tentativo fallito, sta riprovando
turn degraded (model_unreachable)	tutti i tentativi falliti, risposta di cortesia
turn degraded (quota_exhausted)	la quota del modello è finita: c'è scritto per quanto
turn degraded (empty_answer) (tool_loop_ceiling)	gli altri due modi in cui un turno può ripiegare
Groq rate limit is longer than retrying can absorb	limite lungo (di solito quello giornaliero): rinuncia subito invece di insistere
could not read the history, answering without it	database illeggibile: ha risposto lo stesso, senza contesto
the answer was delivered but not remembered	risposta consegnata ma non salvata: la prossima volta non la ricorderà
health probe could not read the conversation store	/health ora risponde 503
database integrity check FAILED ...	database corrotto: è partito il recupero automatico (paragrafo 6.7)
RECOVERED: history restored from ...	cronologia ripristinata da uno snapshot

Quanto spazio occupa il journal e come limitarlo:


```
journalctl --disk-usage
sudo journalctl --vacuum-time=30d      # tiene solo gli ultimi 30 giorni
```

7.2 6.2 La regola d'oro: prima il backup

Nessun aggiornamento di codice o di dipendenze senza uno snapshot precedente. Non è una raccomandazione, è il primo passo obbligatorio di ogni procedura di questo capitolo. Ci vogliono dieci secondi e ti separa da un pomeriggio di ricostruzione.

```
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 20 --no-pager    # controlla che sia andato bene
ls -lht /mnt/backup/emma/ | head -3                  # l'archivio più recente è di adesso
```

Se il backup fallisce, **fermati**: risolvi quello prima di toccare qualunque altra cosa.

7.3 6.3 Aggiornare il codice: PC di sviluppo → GitHub → server

Il flusso è sempre lo stesso e va in una sola direzione. **Sul server non si modifica mai il codice a mano**: se lo facessi, il git pull successivo entrerebbe in conflitto e ti troveresti due versioni divergenti senza sapere quale è quella buona.

7.3.1 Sul PC di sviluppo (Windows)

```
# 1. Snapshot Locale, indipendente da Git
powershell -ExecutionPolicy Bypass -File .\scripts\backup-dev.ps1

# 2. Le modifiche, con le verifiche
ruff format .
ruff check .
pytest

# 3. Commit e push
git add -A
git commit -m "descrizione di cosa cambia e perché"
git push
```

Se la modifica è una release, aggiorna CHANGELOG.md, incrementa la versione secondo semver e aggiungi il tag:

```
git tag -a v0.1.1 -m "v0.1.1 - descrizione breve"
git push --tags
```

Il criterio semver, in breve: **patch** (0.1.x) per correzioni che non cambiano il comportamento; **minor** (0.x.0) per funzionalità nuove compatibili; **major** (x.0.0) per cambiamenti che rompono la compatibilità — nel nostro caso, tipicamente una variabile .env rinominata o rimossa.

7.3.2 Sul server

```
# 1. BACKUP OBBLIGATORIO
sudo systemctl start emma-backup.service
journalctl -u emma-backup.service -n 20 --no-pager

# 2. Annota la versione attuale, per poter tornare indietro
cd /opt/emma
git rev-parse --short HEAD      # per esempio a1b2c3d - segnatelo

# 3. Scarica le modifiche
sudo -u emma git -C /opt/emma pull
```

```
# 4. Aggiorna le dipendenze, se requirements.txt è cambiato
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
```

```
# 5. Verifica prima di riavviare
sudo -u emma /opt/emma/.venv/bin/python -m pytest
```

```
# 6. Riavvia
sudo systemctl restart emma.service
systemctl status emma.service
curl -s http://127.0.0.1:8000/health
```

Se il `git pull` ha toccato i file in `systemd/`, ricopiali e ricarica prima di riavviare — il `git pull` aggiorna il repository, non le unit già installate in `/etc`:

```
sudo cp /opt/emma/systemd/*.service /opt/emma/systemd/*.timer /etc/systemd/system/
sudo systemctl daemon-reload
```

Aggiornamento da una versione precedente alla 0.2.1. Servono due cose che prima non c'erano, entrambe una volta sola:

```
sudo apt install -y sqlite3 # per lo snapshot del backup
sudo -u emma mkdir -p /opt/emma/data && sudo chmod 700 /opt/emma/data
```

e la copia delle unit qui sopra, perché `emma.service` dichiara ora `ReadWritePaths=/opt/emma/data`: senza, il servizio non riesce a scrivere la cronologia.

Verifica finale: scrivi al bot dal telefono. Un servizio `active (running)` non dimostra che l'assistente risponde; un messaggio sì.

Se qualcosa va storto, il paragrafo 6.6 spiega come tornare indietro.

7.4 6.4 Aggiornare le dipendenze bloccate

Le versioni in `requirements.txt` sono fissate apposta. Aggiornarle è un'azione deliberata, da fare sul PC di sviluppo, un pacchetto alla volta.

```
# Cosa è invecchiato
pip list --outdated

# Aggiorna una libreria alla volta, non tutte insieme
pip install --upgrade anthropic
pip show anthropic | Select-String Version # prendi il numero esatto
# scrivi quel numero in requirements.txt

# Verifica
pytest
ruff check .
python main.py # provalo davvero: scrivi al bot dal telefono

git add requirements.txt
git commit -m "bump anthropic to X.Y.Z"
```

Perché una alla volta: se qualcosa si rompe, sai immediatamente quale libreria è stata. Aggiornandone cinque insieme passeresti un'ora a scoprirlo.

Ogni tanto conviene rigenerare l'ambiente da zero, per accorgersi di una dipendenza che avevi installato a mano e che non è in `requirements.txt`:

```
Remove-Item -Recurse -Force .venv
python -m venv .venv
```

```
./.venv\Scripts\pip install -r requirements.txt
pytest
```

Aggiornamenti di sicurezza del sistema operativo — separati e più semplici:

```
sudo apt update && sudo apt upgrade -y
sudo systemctl status emma.service      # controlla che sia sopravvissuto
```

Se l'upgrade tocca il kernel, pianifica un riavvio: `emma.service` è abilitato, quindi riparte da solo.

7.5 6.5 Backup della configurazione e della memoria

Due file **non** sono in Git e vanno protetti dal backup: il `.env`, senza il quale l'assistente non parte, e il database, che contiene tutte le tue conversazioni. Il `virtualenv` invece è escluso apposta, perché si ricostruisce da `requirements.txt`. È per questi due file che la directory dei backup ha permessi `700`.

Il database non viene archiviato copiandolo: `backup.sh` ne fa uno snapshot con `VACUUM INTO`, che produce una copia consistente **mentre il servizio scrive**, poi ne verifica l'integrità e solo allora lo include. Una copia normale, presa con `tar` a servizio acceso, può catturare una transazione a metà: l'archivio si apre senza errori e il database dentro non si apre affatto — un guasto che si scopre solo il giorno del ripristino.

Nell'archivio lo snapshot si chiama `emma.db` e sta accanto a `MANIFEST.txt`, non dentro `data/` (che è escluso dal `tar` proprio per questo). Sono esclusi anche il `virtualenv` e `~/ .cache`: la directory di installazione è anche la home dell'utente `emma`, quindi ci finisce la cache di `pip`, che pesa decine di megabyte ed è interamente ricostruibile da `requirements.txt`. Un archivio sano pesa qualche centinaio di kilobyte; se ne vedi uno da decine di megabyte, qualcosa di non necessario ci è entrato dentro.

Il manifesto dichiara sempre com'è andata:

```
tar -xzOf /mnt/backup/emma/emma-*.tar.gz MANIFEST.txt | grep database
# database:      emma.db (consistent snapshot, integrity verified)
```

Se invece leggi `NOT INCLUDED`, quell'archivio contiene codice e `.env` ma non la cronologia: il motivo è scritto sulla stessa riga (di solito `sqlite3` non installato). **Non è un backup fallito** — il resto è valido — ma va sistemato.

Se vuoi una copia a parte, per esempio prima di rigenerare la chiave API:

```
sudo cp /opt/emma/.env /mnt/backup/emma/env-$(date +%Y%m%d).bak
sudo chmod 600 /mnt/backup/emma/env-*.bak
```

Non copiarlo in Documenti, non mandartelo via mail, non metterlo in un servizio cloud non cifrato: è una chiave a pagamento e il controllo del tuo bot.

7.6 6.6 Ripristino

Un backup mai provato in ripristino non è un backup. Provalo una volta, oggi, quando non serve: scoprire che non funziona mentre serve è tutt'altra esperienza.

7.6.1 6.6.1 Tornare a una versione precedente del codice (senza toccare i backup)

È il caso più frequente: un aggiornamento ha rotto qualcosa e vuoi tornare a prima.

```
cd /opt/emma
git log --oneline -10      # la cronologia: il commit buono è lì
sudo -u emma git checkout a1b2c3d # l'hash annotato al passo 2 del paragrafo 6.3
sudo systemctl restart emma.service
```

Sei ora in *detached HEAD*: va benissimo come misura temporanea. Per tornare all'ultima versione, `sudo -u emma git checkout main`. Se il commit rotto è già su GitHub, la soluzione pulita è correggerlo sul PC di sviluppo con un nuovo commit (o un `git revert`) e rifare il ciclo del paragrafo 6.3 — **non riscrivere la cronologia già pubblicata**, perché romperebbe la copia sul server.

7.6.2 6.6.2 Ripristino completo da un archivio

Serve quando il ripristino da Git non basta: `.env` perso, directory danneggiata, o macchina nuova.

```
# 1. Scegli l'archivio e guarda cosa contiene
ls -lht /mnt/backup/emma/
tar -xzf /mnt/backup/emma/emma-20260829-033012.tar.gz MANIFEST.txt

# 2. Ferma il servizio
sudo systemctl stop emma.service

# 3. Estrai in una directory temporanea (mai direttamente sopra l'installazione)
mkdir -p /tmp/restore
tar -xzf /mnt/backup/emma/emma-20260829-033012.tar.gz -C /tmp/restore
ls /tmp/restore/emma

# 4. Metti da parte l'installazione attuale invece di cancellarla
sudo mv /opt/emma /opt/emma.rotto-$(date +%Y%m%d)

# 5. Rimetti a posto il ripristino
sudo mv /tmp/restore/emma /opt/emma
sudo chown -R emma:emma /opt/emma
sudo chmod 750 /opt/emma
sudo chmod 600 /opt/emma/.env

# 5b. Rimetti la cronologia: nell'archivio lo snapshot sta accanto al manifesto,
#     non dentro data/, quindi va copiato a mano. La directory deve esistere
#     comunque, anche senza cronologia da rimettere: la unit la esige.
sudo -u emma mkdir -p /opt/emma/data
sudo chmod 700 /opt/emma/data
sudo -u emma cp /tmp/restore/emma.db /opt/emma/data/emma.db # se l'archivio ce l'ha

# 6. Ricrea il virtualenv: nell'archivio non c'è, per scelta
sudo -u emma python3 -m venv /opt/emma/.venv
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt

# 7. Riparti
sudo systemctl start emma.service
systemctl status emma.service
curl -s http://127.0.0.1:8000/health
```

Verifica: scrivi al bot dal telefono. Solo allora il ripristino è concluso. Quando sei sicuro che tutto funzioni, elimina `/opt/emma.rotto-*`.

7.6.3 6.6.3 Ripristino su una macchina nuova

Stessa procedura, con il capitolo 1 davanti (utente, directory, pacchetti, disco di backup) e i passi 4.6 e 4.7 dietro, per reinstallare le unit systemd. Il contenuto dell'archivio ti restituisce codice, `.env` e personalità: il resto è sistema, e il sistema si ricostruisce da questa guida.

7.6.4 6.6.4 Ripristino dal PC di sviluppo

Gli zip in `D:\EmmaBackups` contengono il progetto **compresa la directory `.git`**, quindi ognuno è un repository completo con tutta la cronologia. Se il repository locale si corrompe:

1. rinomina la cartella di progetto attuale (non cancellarla);
2. estrai lo zip più recente al suo posto;
3. `git status` e `git log --oneline -5` per verificare che la cronologia ci sia;
4. `git push` per riallineare GitHub, se serve.

7.7 6.7 Problemi comuni

7.7.1 Il bot non risponde

Nell'ordine:

```
systemctl status emma.service           # 1. il servizio è vivo?
journalctl -u emma -n 50 --no-pager     # 2. cosa dicono i log?
curl -s http://127.0.0.1:8000/health    # 3. il processo risponde?
```

- Il servizio non è attivo → guarda l'errore nei log e vai al caso pertinente qui sotto.
- Il servizio è attivo ma nei log non compare `incoming message` → il messaggio non arriva affatto. Stai scrivendo al bot giusto? Il `TELEGRAM_BOT_TOKEN` è quello di *quel* bot?
- Nei log c'è `ignored message from user_id=NNN (not in whitelist)` → è il caso più comune in assoluto. Quel numero `NNN` è il tuo vero user ID: mettilo in `TELEGRAM_ALLOWED_USER_ID` e riavvia. (I log ti hanno appena detto la risposta.)
- C'è `incoming message ma non answered` → il problema è verso Anthropic: cerca `anthropic call failed`.

7.7.2 I messaggi arrivano ma EMMA non risponde mai

Sintomo insidioso: il servizio è active, i log mostrano `incoming message`, ma nessun `answered` — e dal telefono è indistinguibile da un bot spento.

```
journalctl -u emma --since "30 min ago" | grep -E "TimedOut|incoming|answered"
```

Se vedi `telegram.error.TimedOut` con `httpcore.ConnectTimeout` su `connect_tcp`, il processo non riesce ad aprire **nuove** connessioni verso Telegram, mentre la connessione del long polling — già stabilita — continua a funzionare. Per questo i messaggi entrano e le risposte no.

Prima di tutto escludi che sia la macchina:

```
sudo -u emma curl -s -o /dev/null -w "%{http_code} in %{time_total}s\n" \
--max-time 15 https://api.telegram.org/
```

Se `curl` risponde in una frazione di secondo, la rete è a posto e il problema è il pool di connessioni del processo, rimasto appeso dopo un disturbo di rete. **Si risolve con un riavvio**, che lo ricrea da zero:

```
sudo systemctl restart emma.service
```

Attenzione a come misuri se è risolto. `journalctl --since "10 min ago"` può risalire a prima del riavvio e farti ricontare i vecchi errori, facendoti credere che il problema persista. Usa l'istante del riavvio:

```
R=$(systemctl show emma.service -p ActiveEnterTimestamp --value)
journalctl -u emma --since "$R" | grep -c TimedOut
```

Se ricapita spesso, la causa di fondo è la rete verso Telegram. Misurala:

```
for i in $(seq 1 20); do
  sudo -u emma curl -s -o /dev/null --max-time 6 https://api.telegram.org/ \
  && echo -n . || echo -n X
done; echo
```

Su un server solo IPv6 una percentuale di fallimenti è normale: c'è un unico indirizzo raggiungibile e nessun IPv4 su cui ripiegare. Con il codice attuale un invio fallito uccide il turno in silenzio, quindi quella percentuale è anche la quota di messaggi che resteranno senza risposta.

7.7.3 configuration error: required environment variable ... is missing

Il `.env` manca, è nel posto sbagliato o la variabile è vuota.

```
ls -l /opt/emma/.env
sudo grep -c . /opt/emma/.env      # non deve essere 0
```

Il file deve stare nella directory del progetto, accanto a `main.py`.

7.7.4 configuration error: TELEGRAM_ALLOWED_USER_ID must be an integer

Hai messo lo username invece del numero. Chiedi a [@userinfobot](#) il tuo Id e usa quello, senza chiocciola e senza virgolette.

7.7.5 Ricevi sempre “Non riesco a contattare il cervello”

L'API Anthropic non è raggiungibile o rifiuta la chiave. Guarda i log:

```
journalctl -u emma | grep "anthropic call failed" | tail -5
```

- `AuthenticationError / 401` → la chiave è sbagliata, scaduta o revocata. Ne crei una nuova sulla console, la scrivi nel `.env`, `systemctl restart emma`.
- `PermissionDeniedError / 403`, o messaggi su credito → controlla il credito e i limiti di spesa sulla console.
- `APIConnectionError` → problema di rete o DNS del server:

```
curl -sI https://api.anthropic.com | head -1  
resolvectl query api.anthropic.com
```

- `RateLimitError / 429` → **in questo caso non ricevi questa frase:** dalla 0.3.0 la quota ha un messaggio suo (“*Ho raggiunto il limite di richieste verso il modello*”), con il tempo di attesa quando il server lo dichiara. EMMA ritenta da sola i limiti brevi — quelli al minuto, che si liberano in pochi secondi — e rinuncia subito quando l'attesa richiesta è più lunga di quanto i tentativi possano coprire, perché insistere renderebbe solo più lento un rifiuto già deciso. Sul piano gratuito di Groq il tetto è **giornaliero** (200.000 token): in quel caso non c'è niente da fare fino al reset.

```
journalctl -u emma | grep "rate limit" | tail -5
```

Con `LLM_PROVIDER=groq` valgono gli stessi controlli, cercando `groq call failed` invece di `anthropic call failed`. Un 404 sul nome del modello significa che `GROQ_MODEL` non è disponibile per il tuo account: elenca quelli accessibili con il comando del paragrafo 2.8.

7.7.6 Il servizio riparte in continuazione

```
journalctl -u emma -n 100 --no-pager | grep -i error
```

Quasi sempre è un errore di configurazione che si ripete a ogni avvio. Dopo cinque tentativi in cinque minuti `systemd` si ferma da solo: risolto il problema, `sudo systemctl reset-failed emma.service && sudo systemctl start emma.service`.

7.7.7 Il backup non parte o fallisce

```
journalctl -u emma-backup.service -n 30 --no-pager  
systemctl list-timers emma-backup.timer  
findmnt /mnt/backup # il disco è montato?  
sudo -u emma df -h /mnt/backup # c'è spazio?
```

- **warning: ... is not on a separate disk** → non è un errore: il secondo disco non è montato e il backup è finito su `/var/backups/emma`. Se ti aspettavi il disco esterno, `sudo mount -a` e controlla `/etc/fstab` (paragrafo 1.7.3); poi il backup successivo tornerà da solo sul disco giusto.
- **no destination left, no backup taken** → nemmeno il ripiego è scrivibile. Controlla che `/var/backups` esista e che emma possa scriverci, e che `ReadWritePaths=` nella unit copra la destinazione (paragrafo 4.7).
- **Permission denied** → o la directory non appartiene a emma, o manca `ReadWritePaths=` nella unit (paragrafo 4.7).
- **Il timer non compare in list-timers** → non è abilitato: `sudo systemctl enable --now emma-backup.timer`.

7.7.8 EMMA non ricorda più niente, oppure il servizio non parte per il database

Dalla v0.2 la cronologia sta in data/emma.db. Controlla che il file esista e che l'utente emma possa scriverci:

```
ls -l /opt/emma/data/  
sudo -u emma sqlite3 /opt/emma/data/emma.db "SELECT COUNT(*) FROM messages;"
```

- **Failed to set up mount namespaces** e il servizio non parte affatto → la directory dichiarata in ReadWritePaths= non esiste. Creala e riavvia:

```
sudo -u emma mkdir -p /opt/emma/data && sudo chmod 700 /opt/emma/data  
sudo systemctl restart emma.service
```

- **cannot create the database directory ...** nei log → il servizio parte ma non può scrivere. Sotto systemd è quasi sempre ReadWritePaths= in emma.service che non copre il percorso di MEMORY_DB_PATH: allinea le due cose e ricarica con sudo systemctl daemon-reload.
- **unable to open database file** nei log → stesso problema di permessi, oppure MEMORY_DB_PATH nel .env punta a un percorso non scrivibile: sudo chown -R emma:emma /opt/emma/data.
- **database is locked** → due processi EMMA stanno girando insieme. Controlla con systemctl status emma.service e chiudi quello avviato a mano.
- **Il file c'è ma la cronologia è vuota** → normale dopo una cancellazione o al primo avvio; si ripopola dal messaggio successivo.

7.7.9 EMMA ha ripristinato la memoria da sola

Se il database si corrompe, EMMA se ne accorge all'avvio e si ripara. Non è silenziosa: cerca nei log.

```
journalctl -u emma | grep -E "integrity check FAILED|RECOVERED|corrupt"  
ls -l /opt/emma/data/
```

Cosa vedrai, e cosa significa:

Riga	Significato
database integrity check FAILED for ...	il database era danneggiato
corrupt database kept for inspection at ...	il file rotto è lì, non è stato cancellato
RECOVERED: history restored from ...	ripristinato dallo snapshot; i messaggi scritti dopo quello snapshot sono persi
snapshot ... is unusable, trying the one before it	la generazione più recente era rotta, si è usata la precedente
no healthy snapshot available	nessuno snapshot valido: EMMA è ripartita con cronologia vuota

Il file danneggiato resta in data/emma.db.corrotto-<data>. Puoi provare a recuperarci qualcosa:

```
sudo -u emma sqlite3 /opt/emma/data/emma.db.corrotto-20260831-143002 \  
".recover" > /tmp/recuperato.sql
```

Quando hai finito, cancellalo: non serve a nessuno e occupa spazio.

Se succede più di una volta, il problema non è SQLite ma il disco. Controlla dmesg -T | grep -i -E "i/o error|ata" e lo stato SMART (smartctl -a /dev/sda): un database che si corrompe ripetutamente è quasi sempre un supporto che sta morendo, e nessuna auto-riparazione compensa un disco guasto.

7.7.10 Le risposte sono strane o fuori carattere

Hai modificato prompts/system_prompt.txt e non hai riavviato: il prompt viene letto solo all'avvio. sudo systemctl restart emma.service.

7.7.11 git pull dice che ci sono conflitti

Qualcuno — probabilmente tu — ha modificato file direttamente sul server. Per vedere cosa:

```
git -C /opt/emma status
git -C /opt/emma diff
```

Se le modifiche locali non ti servono, sudo -u emma git -C /opt/emma checkout -- . le scarta e poi il pull passa. Se ti servono, portale sul PC di sviluppo e falle rientrare dal flusso normale. E ricorda la regola: sul server non si modifica il codice.

7.8 6.8 Calendario di manutenzione

Quando	Cosa
Automatico, ogni notte	il backup gira alle 3:30
Ogni settimana	un'occhiata a journalctl -u emma -p err --since "7 days ago"
Ogni mese	sudo apt update && sudo apt upgrade; controllo della spesa sulla console Anthropic; ls -lh /mnt/backup/emma/ per verificare che gli archivi ci siano davvero
Ogni tre mesi	pip list --outdated sul PC di sviluppo e aggiornamento ragionato; prova di ripristino (paragrafo 6.6)
Una volta sola, adesso	la prima prova di ripristino, prima che serva

Capitolo 8

Appendice A — Comandi di riferimento

```
# Servizio
sudo systemctl status emma.service
sudo systemctl restart emma.service
sudo systemctl stop emma.service
sudo systemctl start emma.service

# Log
journalctl -u emma -f
journalctl -u emma -n 100 --no-pager
journalctl -u emma --since today -p err

# Salute
curl -s http://127.0.0.1:8000/health

# Backup
sudo systemctl start emma-backup.service
systemctl list-timers emma-backup.timer
ls -lht /mnt/backup/emma/ | head

# Memoria: azzerare la cronologia (snapshot compresi)
sudo systemctl stop emma.service
sudo -u emma rm -f /opt/emma/data/emma.db /opt/emma/data/emma.db.snapshot*
sudo systemctl start emma.service

# Memoria: com'è andato l'ultimo recupero automatico
journalctl -u emma | grep -E "integrity check FAILED|RECOVERED"

# Aggiornamento (dopo il backup)
sudo -u emma git -C /opt/emma pull
sudo -u emma /opt/emma/.venv/bin/pip install -r /opt/emma/requirements.txt
sudo -u emma /opt/emma/.venv/bin/python -m pytest
sudo systemctl restart emma.service

# Cronologia e ritorno indietro
git -C /opt/emma log --oneline -10
sudo -u emma git -C /opt/emma checkout <hash>
sudo -u emma git -C /opt/emma checkout main
```

Capitolo 9

Appendice B — Le variabili di .env

Variabile	Obbligatoria	Default	Note
LLM_PROVIDER	no	anthropic	anthropic o groq
ANTHROPIC_API_KEY	se provider=anthropic	—	comincia con sk-ant-; è un segreto
ANTHROPIC_MODEL	no	claude-sonnet-4-6	qualunque identificativo valido
GROQ_API_KEY	se provider=groq	—	comincia con gsk-; è un segreto
GROQ_MODEL	no	openai/gpt-oss-120b	dipende dal piano account Groq
TELEGRAM_BOT_TOKEN	sì	—	da @BotFather; è un segreto
TELEGRAM_ALLOWED_USER_ID	sì	—	numero, non username; da @userinfobot
MAX_HISTORY_MESSAGES	no	20	messaggi nella finestra; incide sul costo
MEMORY_DB_PATH	no	data/emma.db	file SQLite della storia; creato automaticamente
SYSTEM_PROMPT_PATH	no	prompts/system_prompt.txt	relativo alla directory del progetto
BACKUP_DIR	no	/mnt/backup/emma	letto da backup.sh
BACKUP_KEEP	no	14	archivi conservati dalla rotazione

Nota: il file SQLite (data/emma.db) e i suoi due snapshot contengono la cronologia delle conversazioni e non devono mai finire in un commit Git — .gitignore esclude già l'intera directory data/. Per azzerare la memoria vanno cancellati anche gli snapshot, altrimenti un recupero automatico potrebbe farli tornare: vedi il paragrafo 5.7.

Capitolo 10

Appendice C — Dove guardare quando qualcosa non torna

Domanda	Risposta
Il servizio è vivo?	<code>systemctl status emma.service</code>
Cosa è successo?	<code>journalctl -u emma -n 100 --no-pager</code>
Il processo risponde?	<code>curl -s http://127.0.0.1:8000/health</code>
Quale versione è in esecuzione?	<code>git -C /opt/emma log --oneline -1</code>
Quando è stato l'ultimo backup?	<code>ls -lht /mnt/backup/emma/ \ head -3</code>
Quanto sto spendendo?	<code>journalctl -u emma \ grep -E "(anthropic\ groq) call ok" \ tail -20</code>
Quali sono le impostazioni attive?	il log di avvio: <code>journalctl -u emma \ grep "starting emma"</code>
Quante conversazioni sono in memoria?	<code>sudo -u emma sqlite3 /opt/emma/data/emma.db "SELECT conv_id, COUNT(*) FROM messages GROUP BY conv_id;"</code>
Perché è stato deciso così?	REVISIONE.md, e il capitolo 2 di questa guida

EMMA v0.3.0 — guida aggiornata al 31 agosto 2026. Il sorgente di questo documento è docs/GUIDA.md: modificalo lì e rigenera il PDF, così le due versioni non divergono.